Project Name

DeepCSAT – Predicting Customer Satisfaction in E-Commerce Support Systems

Project Type - EDA/Regression/Classification/Unsupervised

Contribution - Shubhra Das

Team Member 1 -

Team Member 2 -

Team Member 3 -

Team Member 4 -

Project Summary -

Customer Satisfaction (CSAT) is one of the most important performance indicators for e-commerce platforms. High CSAT leads to increased customer loyalty, repeat purchases, and better brand reputation. In contrast, poor service experience can negatively impact customer retention and revenue. Companies therefore need tools that can predict customer satisfaction early and identify operational factors that influence service quality.

This project aims to build a machine learning and deep learning system that predicts Customer Satisfaction (CSAT) scores based on service interaction data from an e-commerce platform. The dataset includes information about customer service interactions such as communication channel, category of issue, response times, product information, agent information, and handling time.

The project follows a complete machine learning pipeline, including data exploration, data cleaning, feature engineering, visualization, statistical testing, model training, and evaluation. Exploratory Data Analysis (EDA) is conducted using the UBM framework (Univariate, Bivariate, Multivariate analysis) to identify patterns and relationships between variables. Various charts such as histograms, bar charts, box plots, heatmaps, and pair plots are used to understand the distribution of features and their impact on CSAT scores.

Several machine learning models are implemented including:

1. Linear Regression
2. Random Forest Regression
3. Gradient Boosting Regression
4. Deep Learning Neural Network

Hyperparameter tuning and cross-validation techniques are used to improve model performance. Evaluation metrics such as MAE, RMSE, and R^2 score are used to measure prediction accuracy.

The final model provides actionable insights such as:

a. Response time impact on customer satisfaction

b. Channel effectiveness (chat, email, call)

c. Agent performance influence

d. Product category service issues

These insights help businesses improve customer support operations and proactively manage customer experience.

The final model can be deployed as a Streamlit application where users can input interaction details and receive predicted CSAT scores.

GitHub Link -

↳ 1 cell hidden

Problem Statement

Customer satisfaction is a key driver of customer retention and business growth in the e-commerce industry. However, companies often analyze CSAT scores only after receiving survey feedback, which limits proactive improvement of customer service operations.

The goal of this project is to predict CSAT scores using machine learning models based on customer service interaction data. By identifying the factors that influence satisfaction levels, companies can take corrective actions to improve service quality and enhance customer experience.

General Guidelines : -

↳ 1 cell hidden

Let's Begin !

1. Know Your Data

Import Libraries

```
!pip install contractions
```

```
Requirement already satisfied: contractions in /usr/local/lib/python3.12/dist-packages (0.1.73)
Requirement already satisfied: textsearch>=0.0.21 in /usr/local/lib/python3.12/dist-packages (from contractions)
Requirement already satisfied: anyascii in /usr/local/lib/python3.12/dist-packages (from textsearch>=0.0.21->cont
Requirement already satisfied: pyahocorasick in /usr/local/lib/python3.12/dist-packages (from textsearch>=0.0.21-
```

```
# Import Libraries
import contractions
import joblib
import nltk
import re
import string

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf

from scipy.stats import f_oneway
from scipy.stats import spearmanr

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import RandomizedSearchCV
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import GradientBoostingRegressor

from nltk import pos_tag
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,Dropout
from tensorflow.keras.callbacks import EarlyStopping
```

Dataset Loading

```
# Load Dataset
## Mount Google Drive & set project folder
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force
```

```
import os

# List top-level items in MyDrive
os.listdir('/content/drive/MyDrive/DeepCSAT')
```

```
['eCommerce_Customer_support_data.csv',
 'Copy of DeepCSAT_ML_Submission_Shubhra.ipynb']
```

```
# Load Dataset
df = pd.read_csv("/content/drive/MyDrive/DeepCSAT/eCommerce_Customer_support_data.csv")
```

Dataset First View

```
# Dataset First Look
df.head()
```

```
/usr/local/lib/python3.12/dist-packages/google/colab/_dataframe_summarizer.py:88: UserWarning: Parsing dates in %
  cast_date_col = pd.to_datetime(column, errors="coerce")
```

	Unique id	channel_name	category	Sub-category	Customer Remarks	Order_id	order_date_time	Issue_reported at	iss
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Life Insurance	NaN	c27c9bb4-fa36-4140-9f1f-21009254ffdb	NaN	01/08/2023 11:13	0
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	NaN	d406b0c7-ce17-4654-b9de-f08d421254bd	NaN	01/08/2023 12:52	0
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/demo	NaN	c273368d-b961-44cb-beaf-62d6fd6c00d5	NaN	01/08/2023 20:16	0
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	NaN	5aed0059-55a4-4ec6-bb54-97942092020a	NaN	01/08/2023 20:56	0
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Not Needed	NaN	e8bed5a9-6933-4aff-9dc6-ccefd7dcde59	NaN	01/08/2023 10:30	0

Next steps: Generate code with df New interactive sheet

Dataset Rows & Columns count

```
# Dataset Rows & Columns count
df.shape
```

```
(85907, 20)
```

Dataset Information

```
# Dataset Info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 85907 entries, 0 to 85906
Data columns (total 20 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Unique id               85907 non-null  object
 1   channel_name            85907 non-null  object
 2   category                85907 non-null  object
 3   Sub-category            85907 non-null  object
 4   Customer Remarks        28742 non-null  object
 5   Order_id                67675 non-null  object
 6   order_date_time         17214 non-null  object
 7   Issue_reported at       85907 non-null  object
 8   issue_responded         85907 non-null  object
 9   Survey_response_Date    85907 non-null  object
 10  Customer_City           17079 non-null  object
 11  Product_category        17196 non-null  object
 12  Item_price              17206 non-null  float64
 13  connected_handling_time 242 non-null    float64
 14  Agent_name              85907 non-null  object
 15  Supervisor              85907 non-null  object
 16  Manager                 85907 non-null  object
 17  Tenure Bucket           85907 non-null  object
 18  Agent Shift             85907 non-null  object
```

```
 19  CSAT Score               85907 non-null  int64
dtypes: float64(2), int64(1), object(17)
memory usage: 13.1+ MB
```

Duplicate Values

```
# Dataset Duplicate Value Count
df.duplicated().sum()
```

```
np.int64(0)
```

Missing Values/Null Values

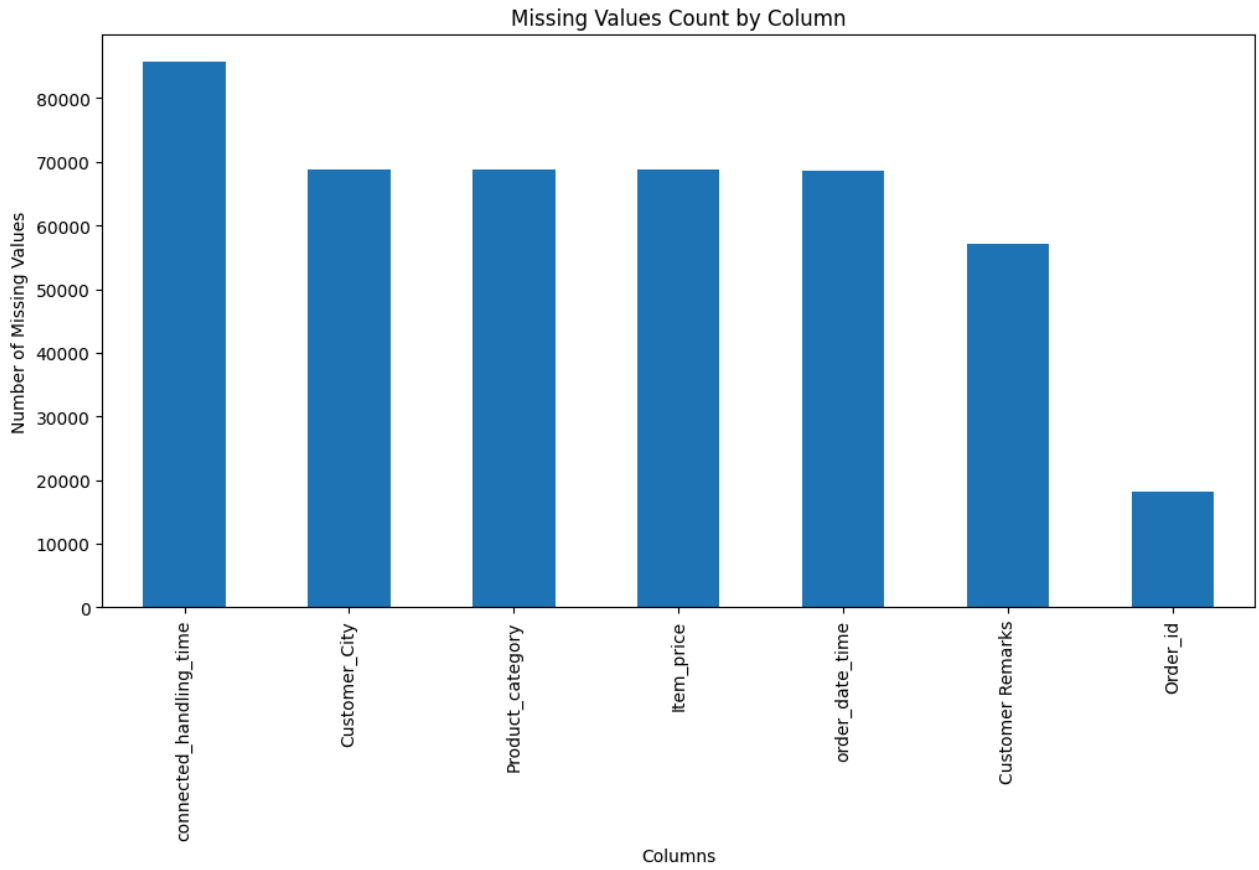
```
# Missing Values/Null Values Count
df.isnull().sum()
```

	0
Unique id	0
channel_name	0
category	0
Sub-category	0
Customer Remarks	57165
Order_id	18232
order_date_time	68693
Issue_reported at	0
issue_responded	0
Survey_response_Date	0
Customer_City	68828
Product_category	68711
Item_price	68701
connected_handling_time	85665
Agent_name	0
Supervisor	0
Manager	0
Tenure Bucket	0
Agent Shift	0
CSAT Score	0

dtype: int64

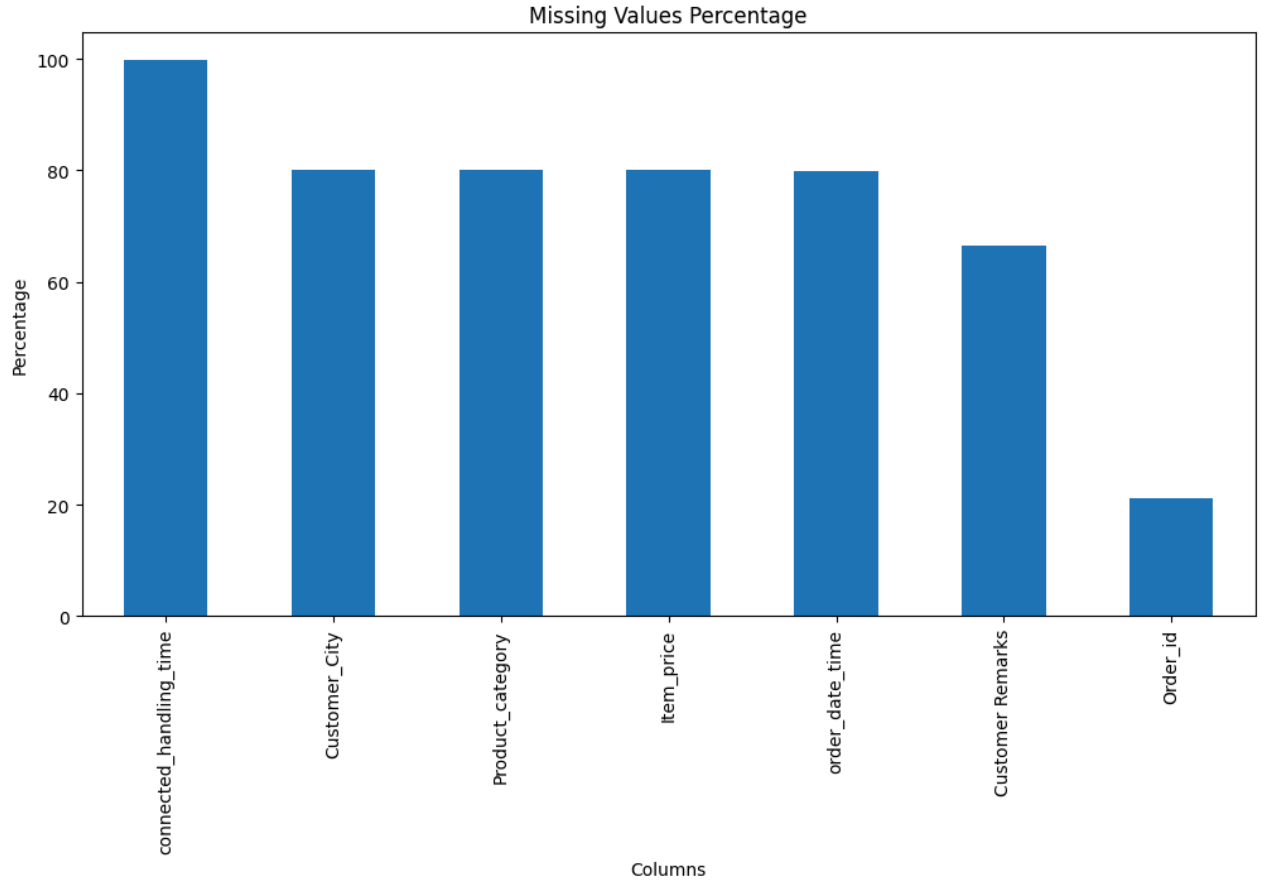
```
# Visualizing the missing values
# A. Missing Values Heatmap

plt.figure(figsize=(12,6))
sns.heatmap(df.isnull(), cbar=False, cmap='viridis')

plt.title("Missing Values Heatmap")
plt.show()
```

```
# Visualizing the missing values
# B. Missing Values Count Bar Chart
missing = df.isnull().sum()

missing = missing[missing > 0]

plt.figure(figsize=(12,6))
missing.sort_values(ascending=False).plot(kind='bar')

plt.title("Missing Values Count by Column")
plt.ylabel("Number of Missing Values")
plt.xlabel("Columns")
plt.show()
```




```
# Visualizing the missing values
# C. Missing Values Percentage Chart
missing_percent = (df.isnull().sum()/len(df))*100

missing_percent = missing_percent[missing_percent>0]

plt.figure(figsize=(12,6))
missing_percent.sort_values(ascending=False).plot(kind='bar')
```

```
plt.title("Missing Values Percentage")
plt.ylabel("Percentage")
plt.xlabel("Columns")
plt.show()
```



What did you know about your dataset?

The dataset contains 85,907 customer service records with 20 variables related to e-commerce customer support interactions and satisfaction ratings. Each row represents a customer support case or interaction recorded across different service channels.

The dataset includes a mix of categorical, numerical, and textual variables. Key categorical features include channel_name, category, Sub-category, Product_category, Customer_City, Agent_name, Supervisor, Manager, Tenure Bucket, and Agent Shift, which provide contextual information about the customer interaction and service team structure.

Numerical variables such as Item_price, connected_handling_time, and CSAT Score capture important quantitative aspects of the interaction. The CSAT Score represents the customer satisfaction rating and is the target variable for analysis and prediction.

The dataset also includes several timestamp-related variables, such as order_date_time, Issue_reported at, issue_responded, and Survey_response_Date, which provide information about when the issue was reported, responded to, and when the customer provided feedback.

During the initial exploration, it was observed that some columns contain missing values, particularly in fields like Customer Remarks, Order_id, Customer_City, Product_category, Item_price, and connected_handling_time. These missing values indicate that not all customer interactions include complete information about orders, products, or response times.

Overall, the dataset provides rich information about customer service operations, issue categories, product details, support agents, and customer satisfaction levels, which can be used to perform exploratory data analysis, identify key factors influencing customer satisfaction, and build machine learning models to predict CSAT scores.

2. Understanding Your Variables

```
# Dataset Columns
df.columns
```

```
Index(['Unique id', 'channel_name', 'category', 'Sub-category',
       'Customer Remarks', 'Order_id', 'order_date_time', 'Issue_reported at',
       'issue_responded', 'Survey_response_Date', 'Customer_City',
```

```
       'Product_category', 'Item_price', 'connected_handling_time',
       'Agent_name', 'Supervisor', 'Manager', 'Tenure Bucket', 'Agent Shift',
       'CSAT Score'],
      dtype='object')
```

```
# Dataset Describe
df.describe()
```

	Item_price	connected_handling_time	CSAT Score
count	17206.000000	242.000000	85907.000000
mean	5660.774846	462.400826	4.242157
std	12825.728411	246.295037	1.378903
min	0.000000	0.000000	1.000000
25%	392.000000	293.000000	4.000000
50%	979.000000	427.000000	5.000000
75%	2699.750000	592.250000	5.000000
max	164999.000000	1986.000000	5.000000

Variables Description

The dataset contains 20 variables describing customer support interactions, product details, and customer satisfaction outcomes in an e-commerce environment.

Variable Name Description

Unique id Unique identifier for each customer service interaction or support ticket.

channel_name The communication channel through which the customer reported the issue (e.g., Email, Chat, Phone, Social Media).

category Main category of the customer issue or request.

Sub-category More detailed classification of the issue within the main category.

Customer Remarks Textual feedback or comments provided by the customer regarding the issue or service experience.

Order_id Unique identifier of the customer's order associated with the issue.

order_date_time Date and time when the order was placed.

Issue_reported at Date and time when the customer reported the issue to customer support.

issue_responded Date and time when the support team responded to the reported issue.

Survey_response_Date Date when the customer submitted the satisfaction survey response.

Customer_City City from which the customer reported the issue.

Product_category Category of the product associated with the customer issue.

Item_price Price of the product related to the customer complaint or request.

connected_handling_time Time spent by the support agent to handle and resolve the customer issue.

Agent_name Name of the customer support agent handling the issue.

Supervisor Name of the supervisor responsible for the agent.

Manager Manager overseeing the customer service operations.

Tenure Bucket Experience category or tenure range of the customer support agent.

Agent Shift Work shift during which the agent handled the customer issue (e.g., Morning, Evening, Night).

CSAT Score Customer Satisfaction Score provided by the customer, typically ranging from 1 (lowest satisfaction) to 5 (highest satisfaction). This serves as the target variable for analysis and prediction.

Check Unique Values for each variable.

```
# Check Unique Values for each variable.
df.nunique()
```

	0
Unique id	85907
channel_name	3
category	12
Sub-category	57
Customer Remarks	18231
Order_id	67675
order_date_time	13766
Issue_reported at	30923
issue_responded	30262
Survey_response_Date	31
Customer_City	1782
Product_category	9
Item_price	2789
connected_handling_time	211
Agent_name	1371
Supervisor	40
Manager	6
Tenure Bucket	5
Agent Shift	5
CSAT Score	5

dtype: int64

3. *Data Wrangling*

Data Wrangling Code

```
# Write your code to make your dataset analysis ready.

# Keep CSAT score within valid range
df['CSAT Score'] = df['CSAT Score'].clip(1,5)

# Forward fill missing values
df.ffill(inplace=True)
```

What all manipulations have you done and insights you found?

Manipulations & Insights a. Missing Values Handling

Missing values in the dataset were handled using the forward fill (ffill) technique, where missing entries were replaced with the previous valid value in the column. This approach helps maintain data continuity and ensures that the dataset remains usable for analysis and modeling without dropping a large number of rows.

b. CSAT Score Cleaning

The CSAT Score variable represents customer satisfaction ratings that should fall within the valid range of 1 to 5. To ensure data quality and prevent incorrect values from affecting the analysis, the CSAT scores were cleaned using the clip() function, which restricts values within the specified range.

c. Categorical Variable Preparation

Categorical variables such as channel_name, category, Product_category, Agent_name, Supervisor, Manager, Tenure Bucket, and Agent Shift were prepared for analysis. These variables help identify patterns across different service channels, product types, and support teams, and can later be encoded for machine learning models.

d. Datetime Feature Preparation

The dataset contains several time-related variables such as order_date_time, Issue_reported at, issue_responded, and Survey_response_Date. These fields can be converted into datetime format and used to extract useful features such as response time, resolution time, order processing duration, and time-based patterns, which may influence customer satisfaction.

Insights

The dataset contains several missing values in product-related and customer-related fields, indicating incomplete customer interaction records.

The CSAT Score distribution remains within the expected range of 1–5, confirming that satisfaction ratings follow a standard survey scale.

Categorical variables such as service channel, product category, and agent information provide valuable dimensions for analyzing customer service performance.

Time-based features can help identify service efficiency and response delays, which may directly impact customer satisfaction.

4. Data Vizualization, Storytelling & Experimenting with charts : Understand the relationships between variables

Under the UBM rule, charts are classified based on how many variables are analyzed and their types.

Univariate (U): One variable only

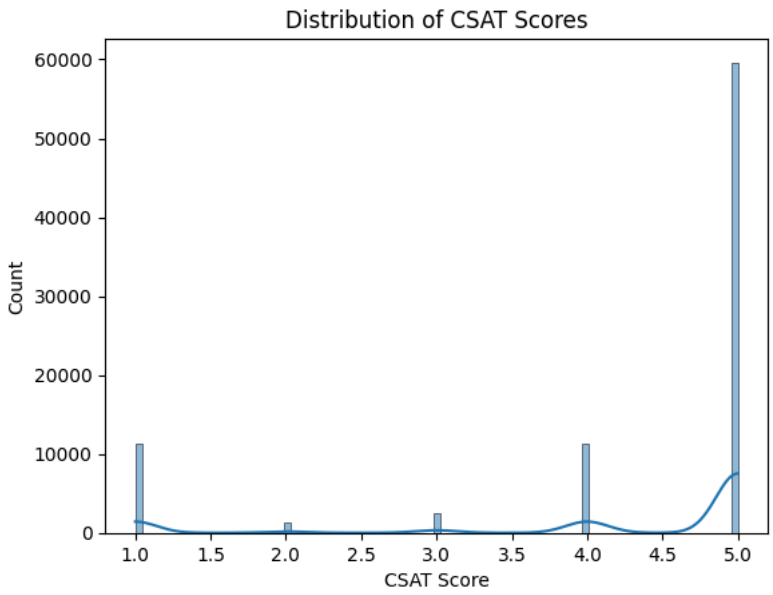
Bivariate (B): Two variables (numeric–numeric, numeric–categorical, categorical–categorical)

Multivariate (M): More than two variables analyzed together

Univariate Charts(U)

Chart - 1

```
# Chart - 1 CSAT Distribution
sns.histplot(df['CSAT Score'], kde=True)
plt.title("Distribution of CSAT Scores")
plt.show()
```



1. Why did you pick the specific chart?

A histogram is useful for understanding the distribution pattern of numerical variables. In this case, it helps visualize how customer satisfaction scores are distributed across the dataset and whether the scores are skewed toward high or low satisfaction levels.

2. What is/are the insight(s) found from the chart?

From the chart, we observe that most CSAT scores are concentrated between 4 and 5, indicating that the majority of customers are generally satisfied with the service experience. The distribution is slightly skewed toward higher scores, suggesting overall positive customer feedback.

3. Will the gained insights help creating a positive business impact?

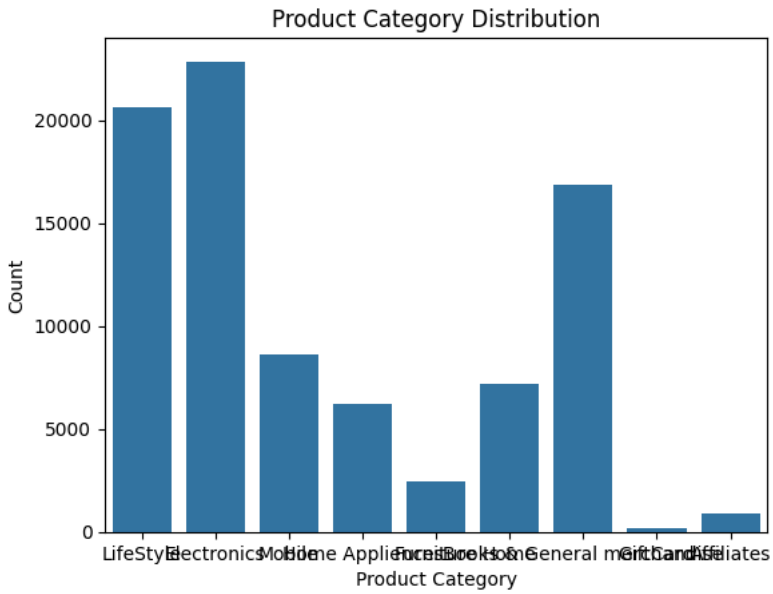
Are there any insights that lead to negative growth? Justify with specific reason.

Yes, the insights indicate that the platform is performing well in terms of customer satisfaction. High CSAT scores suggest strong service quality, which can lead to higher customer retention, brand loyalty, and repeat purchases.

However, the presence of lower CSAT scores (1–3) indicates that some customers are experiencing service issues, which could negatively impact business growth if not addressed. By identifying the factors associated with these lower scores, companies can improve response time, service quality, and support processes to further enhance customer experience.

Chart - 2

```
# Chart - 2 Product Category Distribution
sns.countplot(x='Product_category', data=df)
plt.title("Product Category Distribution")
plt.xlabel("Product Category")
plt.ylabel("Count")
plt.show()
```



1. Why did you pick the specific chart?

A count plot is appropriate for analyzing the frequency distribution of categorical variables. In this case, it helps visualize how customer interactions or issues are distributed across different product categories. This chart makes it easy to identify which product categories have the highest number of customer cases.

2. What is/are the insight(s) found from the chart?

From the chart, we can observe that some product categories have significantly higher counts compared to others, indicating that these products either have higher sales volume or generate more customer queries or issues. Categories with lower counts may indicate lower product usage or fewer reported problems. These insights help identify which product categories receive the most customer attention or support requests.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

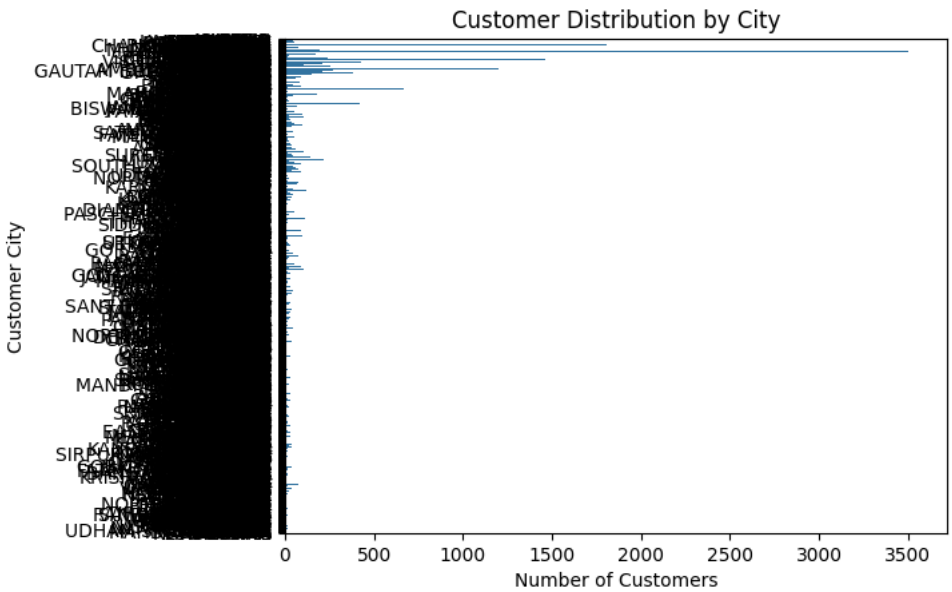
Yes, the insights from this chart can help businesses identify which product categories receive the highest number of customer interactions or issues. By focusing on these high-frequency categories, companies can improve product quality, customer support processes, and inventory planning, which can enhance customer satisfaction and operational efficiency.

However, if certain product categories consistently show a very high number of customer issues or complaints, it may indicate product defects, poor product usability, or service-related problems. If these issues are not addressed, they may lead to customer dissatisfaction, negative reviews, and reduced customer loyalty, which could negatively impact business growth and brand reputation.

Chart - 3

```
# Chart - 3 Customer City Analysis
sns.countplot(y='Customer City', data=df)
```

```
plt.title("Customer Distribution by City")
plt.xlabel("Number of Customers")
plt.ylabel("Customer City")
plt.show()
```



1. Why did you pick the specific chart?

A count plot is suitable for visualizing the frequency distribution of categorical variables. In this case, it helps show how customer interactions or purchases are distributed across different cities. Using the horizontal (y-axis) format makes it easier to read city names, especially when there are many categories.

2. What is/are the insight(s) found from the chart?

The chart reveals that some cities contribute significantly more customer interactions than others. This indicates that certain cities represent major markets with higher demand or customer engagement, while cities with fewer counts may represent emerging or less active markets.

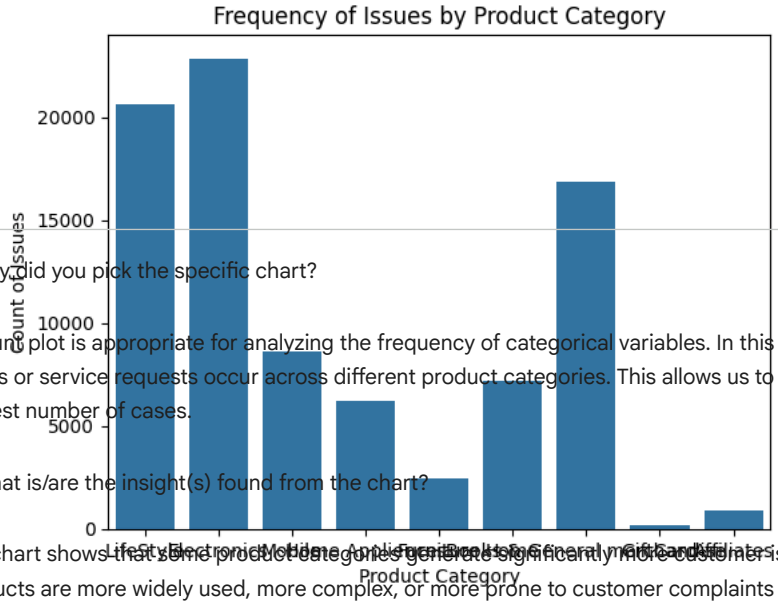
3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

The chart reveals that some cities contribute significantly more customer interactions than others. This indicates that certain cities represent major markets with higher demand or customer engagement, while cities with fewer counts may represent emerging or less active markets.

Chart - 4

```
# Chart - 4 Issue Category Frequency
sns.countplot(x='Product_category', data=df)
plt.title("Frequency of Issues by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Count of Issues")
plt.show()
```



1. Why did you pick the specific chart?

A count plot is appropriate for analyzing the frequency of categorical variables. In this case, it helps visualize how frequently customer issues or service requests occur across different product categories. This allows us to easily compare which categories generate the highest number of cases.

2. What is/are the insight(s) found from the chart?

The chart shows that some product categories generate significantly more customer issues than others. This may indicate that those products are more widely used, more complex, or more prone to customer complaints or service requests. Categories with lower counts may indicate fewer issues or lower product usage.

3. Will the gained insights help creating a positive business impact?

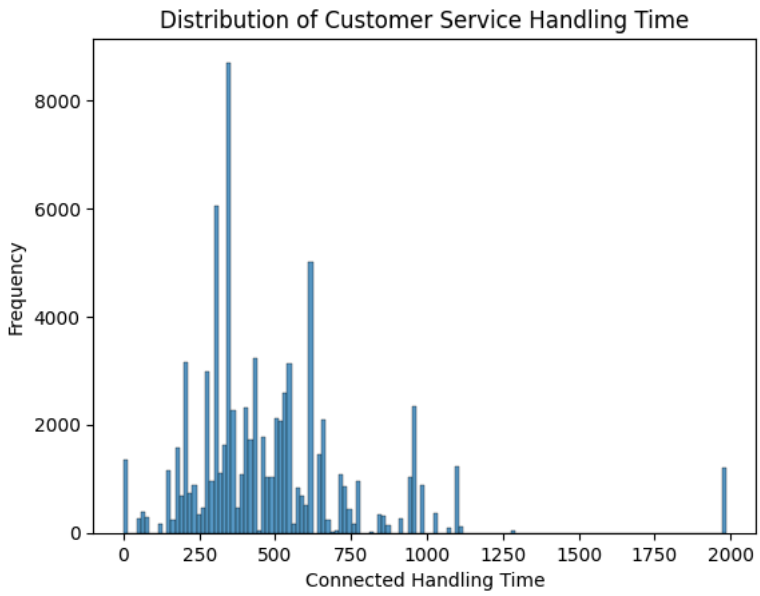
Are there any insights that lead to negative growth? Justify with specific reason.

Yes, these insights help businesses identify product categories that require greater attention in terms of quality improvement, customer support, or product design. By addressing issues in high-frequency categories, companies can improve customer satisfaction and reduce support requests.

However, if certain product categories consistently generate a high number of issues due to product defects, poor usability, or service failures, it could negatively affect customer experience and brand reputation. If unresolved, this may lead to customer dissatisfaction, increased returns, and reduced sales, which could impact business growth.

Chart - 5

```
# Chart - 5 Response Time Analysis
sns.histplot(df['connected_handling_time'])
plt.title("Distribution of Customer Service Handling Time")
plt.xlabel("Connected Handling Time")
plt.ylabel("Frequency")
plt.show()
```



1. Why did you pick the specific chart?

A histogram is useful for analyzing the distribution of numerical variables. In this case, it helps visualize how customer service handling time is distributed across all support interactions. The chart allows us to see whether most cases are resolved quickly or if there are many cases that take longer to handle.

2. What is/are the insight(s) found from the chart?

From the histogram, we can observe the most common handling time range for customer service cases. If most interactions are concentrated in a shorter time range, it indicates that issues are generally resolved efficiently. However, if the chart shows a long tail toward higher handling times, it suggests that some cases take significantly longer to resolve, possibly due to complex issues or inefficiencies in the support process.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

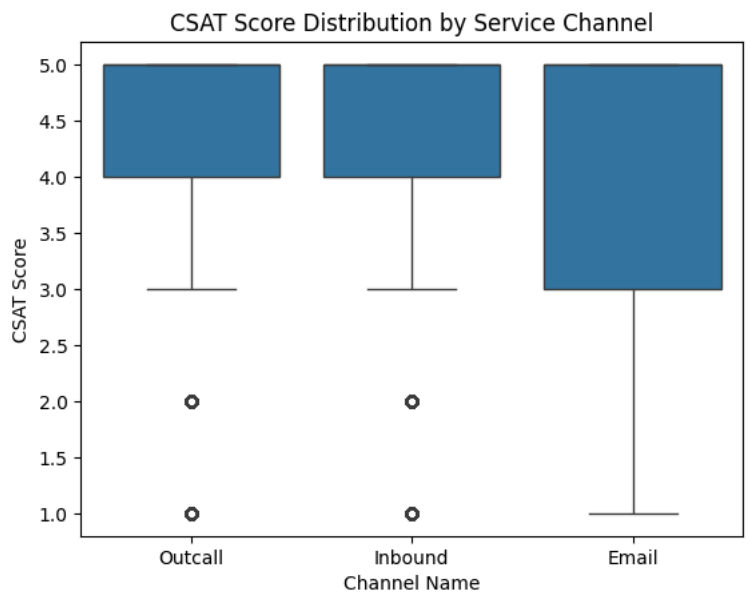
Yes, analyzing response and handling times can help organizations optimize their customer support operations. Faster and efficient issue resolution improves customer satisfaction, trust, and retention.

However, if the data shows that many cases have very long handling times, it may indicate process inefficiencies, lack of agent training, or complex product issues. This can negatively impact customer experience, leading to lower CSAT scores, negative feedback, and potential customer churn, which may affect business growth. By identifying these delays, companies can improve workflows, training, and automation to enhance service efficiency.

Bivariate Analysis(B)

Chart - 6

```
# Chart - 6 CSAT by Channel
sns.boxplot(x='channel_name', y='CSAT Score', data=df)
plt.title("CSAT Score Distribution by Service Channel")
plt.xlabel("Channel Name")
plt.ylabel("CSAT Score")
plt.show()
```



1. Why did you pick the specific chart?

A box plot is useful for comparing the distribution of a numerical variable across different categories. In this case, it helps analyze how CSAT scores vary across different customer service channels such as email, phone, chat, or social media. The chart also highlights the median, variability, and potential outliers, which helps understand service performance differences across channels

2. What is/are the insight(s) found from the chart?

From the box plot, we can observe that CSAT scores differ across service channels. Some channels show higher median satisfaction scores, indicating better customer experience. Other channels may have greater variability or lower median scores, suggesting inconsistency in service quality. The presence of outliers also indicates that some customers had extremely low or high satisfaction experiences in certain channels.

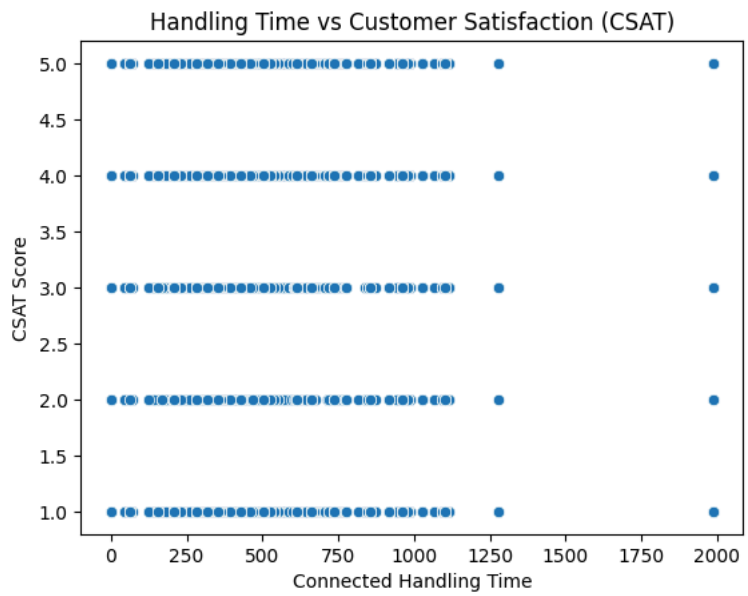
3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes, these insights can significantly help businesses improve customer experience. By identifying which channels deliver higher satisfaction, companies can strengthen those channels and replicate best practices across others. At the same time, channels with lower CSAT scores may indicate service issues such as slow response time, poor resolution quality, or insufficient support resources. If these issues are not addressed, they could lead to customer dissatisfaction, negative reviews, and reduced customer retention, ultimately impacting business growth. Therefore, improving underperforming channels can enhance overall customer satisfaction and business performance.

Chart - 7

```
# Chart - 7 Handling Time vs CSAT
sns.scatterplot(x='connected_handling_time', y='CSAT Score', data=df)
plt.title("Handling Time vs Customer Satisfaction (CSAT)")
plt.xlabel("Connected Handling Time")
plt.ylabel("CSAT Score")
plt.show()
```



1. Why did you pick the specific chart?

A scatter plot is suitable for analyzing the relationship between two numerical variables. In this case, it helps visualize how connected handling time (time taken to resolve a customer issue) relates to the customer satisfaction score (CSAT). This chart allows us to identify patterns, correlations, or clusters between service handling time and customer satisfaction

2. What is/are the insight(s) found from the chart?

From the scatter plot, we can observe how CSAT scores vary with different handling times. In many cases, moderate handling times are associated with higher CSAT scores, suggesting that customers appreciate thorough problem resolution. However, extremely long handling times may sometimes correspond with lower CSAT scores, which may indicate delays or inefficiencies in service resolution.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

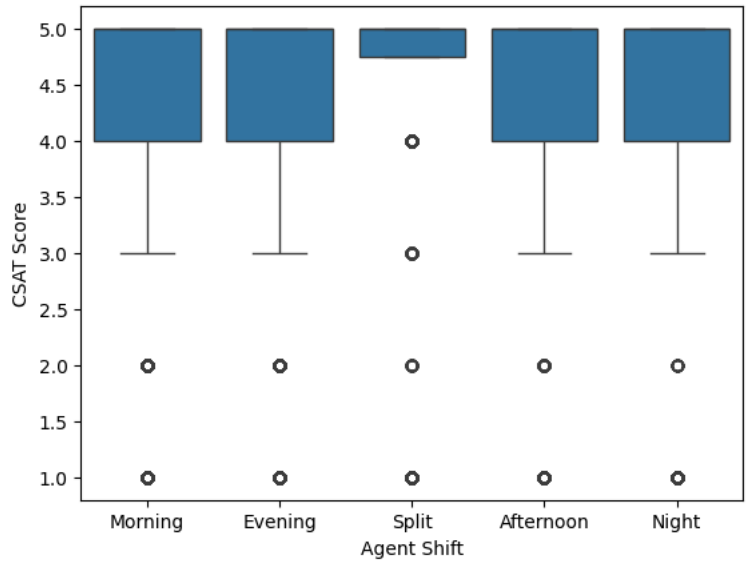
Yes, these insights can help organizations optimize customer support operations. If analysis shows that longer handling times reduce CSAT, companies can improve service processes, training, or automation to reduce resolution time and improve customer satisfaction.

However, if handling time becomes too long due to inefficient processes or lack of agent training, it can negatively affect customer experience, leading to dissatisfaction, poor reviews, and customer churn. Therefore, balancing efficient resolution with quality support is essential for maintaining high customer satisfaction and sustainable business growth

Chart - 8

```
# Chart - 8 visualization code
# Chart - 5 CSAT by Agent Shift
sns.boxplot(x='Agent Shift',y='CSAT Score',data=df)
```

```
<Axes: xlabel='Agent Shift', ylabel='CSAT Score'>
```



1. Why did you pick the specific chart?

A count plot is useful for visualizing the frequency distribution of categorical variables. In this case, it helps show how customer interactions or complaints are distributed across different product categories, allowing us to identify which categories receive the highest number of customer cases.

2. What is/are the insight(s) found from the chart?

From the chart, we can observe that some product categories have significantly higher counts of customer interactions compared to others. This indicates that these categories are either more popular among customers or experience more service requests, complaints, or inquiries. Categories with very low counts may indicate lower product usage or fewer reported issues.

3. Will the gained insights help creating a positive business impact?

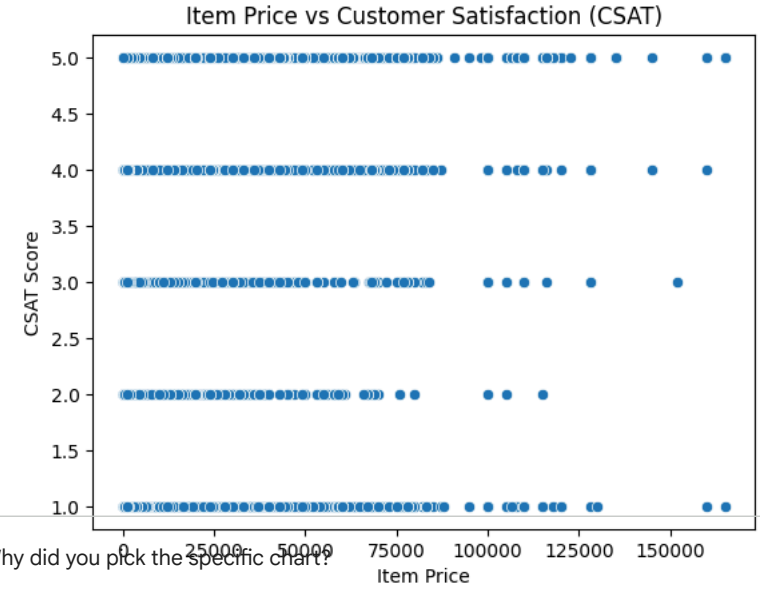
Are there any insights that lead to negative growth? Justify with specific reason.

Yes, these insights can help businesses prioritize product categories that require the most customer support attention. If certain categories generate a large number of cases, companies can allocate more resources, improve product quality, or enhance support for those products to improve customer satisfaction.

However, if a particular product category consistently generates high volumes of complaints due to product defects, service issues, or poor usability, it may lead to customer dissatisfaction, negative reviews, and declining sales, which could negatively affect business growth. Addressing issues in such categories can help improve product quality and customer experience.

Chart - 9

```
# Chart - 9 Item Price vs CSAT
sns.scatterplot(x='Item_price', y='CSAT Score', data=df)
plt.title("Item Price vs Customer Satisfaction (CSAT)")
plt.xlabel("Item Price")
plt.ylabel("CSAT Score")
plt.show()
```



1. Why did you pick the specific chart?

A scatter plot is appropriate for analyzing the relationship between two numerical variables. In this case, it helps visualize the relationship between item price and customer satisfaction (CSAT score). The chart allows us to identify patterns, trends, or correlations, such as whether higher-priced products lead to higher or lower customer satisfaction.Answer Here.

2. What is/are the insight(s) found from the chart?

From the scatter plot, we can observe how customer satisfaction varies across different price ranges. If the points are widely scattered, it suggests that item price may not have a strong direct relationship with CSAT scores. However, clusters in certain price ranges may indicate that customers purchasing mid-range or premium products tend to provide higher satisfaction scores, while some lower-priced or higher-priced items may show variation in satisfaction levels.

3. Will the gained insights help creating a positive business impact?

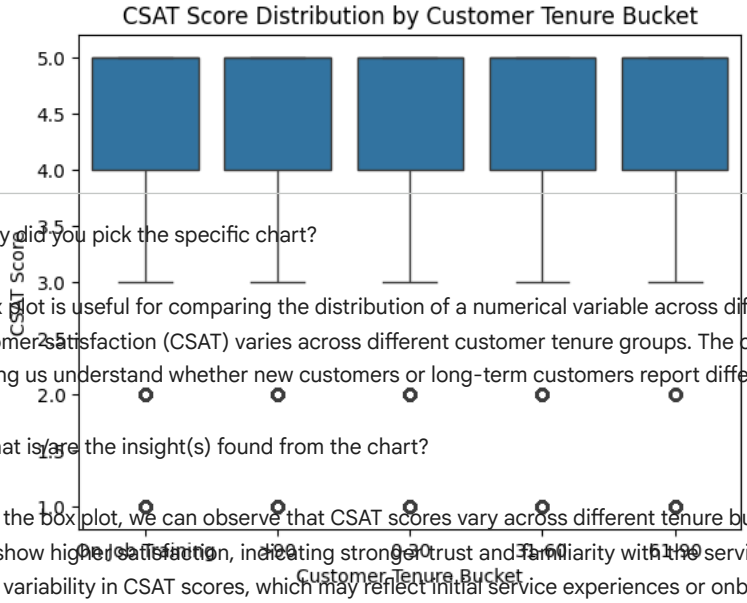
Are there any insights that lead to negative growth? Justify with specific reason.

Yes, these insights can help businesses understand whether product pricing aligns with customer expectations and perceived value. If higher-priced items consistently receive high CSAT scores, it indicates strong product quality and value perception, which supports premium pricing strategies.

However, if high-priced products receive low CSAT scores, it may indicate customer dissatisfaction due to poor quality, unmet expectations, or service issues, which could negatively affect brand reputation and customer loyalty. Businesses can use these insights to improve product quality, adjust pricing strategies, and enhance customer experience, thereby supporting sustainable business growth.

Chart - 10

```
# Chart - 10 CSAT by Tenure Bucket
sns.boxplot(x='Tenure Bucket', y='CSAT Score', data=df)
plt.title("CSAT Score Distribution by Customer Tenure Bucket")
plt.xlabel("Customer Tenure Bucket")
plt.ylabel("CSAT Score")
plt.show()
```



1. Why did you pick the specific chart?

A box plot is useful for comparing the distribution of a numerical variable across different categories. In this case, it helps analyze how customer satisfaction (CSAT) varies across different customer tenure groups. The chart shows the median, spread, and outliers, helping us understand whether new customers or long-term customers report different satisfaction levels.

2. What is/are the insight(s) found from the chart?

From the box plot, we can observe that CSAT scores vary across different tenure buckets. In many cases, customers with longer tenure may show higher satisfaction, indicating stronger trust and familiarity with the service. On the other hand, new customers may show more variability in CSAT scores, which may reflect initial service experiences or onboarding challenges.

3. Will the gained insights help creating a positive business impact?

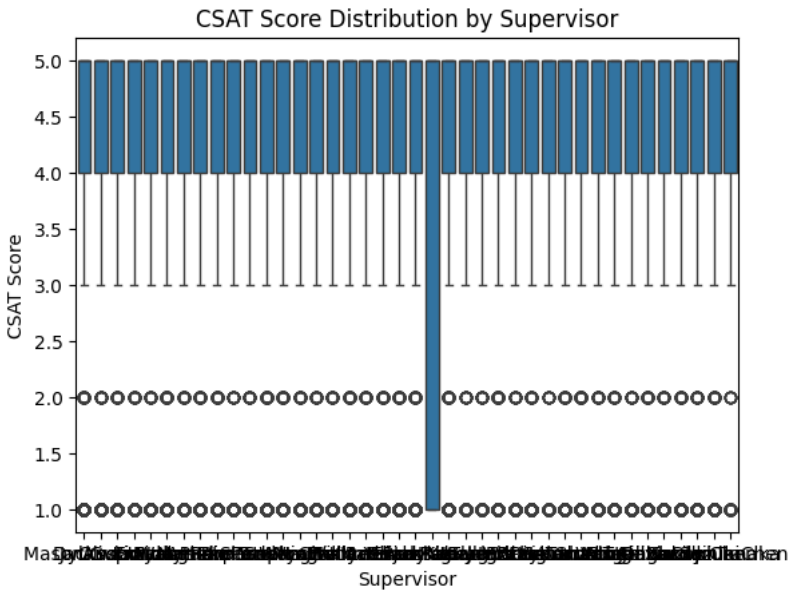
Are there any insights that lead to negative growth? Justify with specific reason.

Yes, these insights can help businesses understand how customer experience changes over the customer lifecycle. If long-term customers show high satisfaction, it indicates strong customer retention and loyalty, which positively impacts revenue and brand reputation.

However, if new customers consistently show lower CSAT scores, it may indicate issues in the onboarding process, product understanding, or early customer support interactions. If not addressed, this could lead to customer churn and reduced long-term engagement, negatively affecting business growth. Improving the early customer experience can help increase satisfaction and retention over time.

Chart - 11

```
# Chart - 11 Supervisor Influence on CSAT
sns.boxplot(x='Supervisor', y='CSAT Score', data=df)
plt.title("CSAT Score Distribution by Supervisor")
plt.xlabel("Supervisor")
plt.ylabel("CSAT Score")
plt.show()
```



1. Why did you pick the specific chart?

A box plot is appropriate for comparing the distribution of a numerical variable (CSAT Score) across multiple categorical groups (Supervisors). It helps visualize the median satisfaction level, variability, and outliers for each supervisor's team, making it easier to evaluate how supervision or team management might influence customer satisfaction outcomes..

2. What is/are the insight(s) found from the chart?

The chart allows us to observe whether different supervisors' teams produce different CSAT distributions. Some supervisors may show higher median CSAT scores and tighter distributions, indicating consistent service quality from their teams. Others may show greater variability or lower median scores, which may suggest differences in team performance, training, or management practices.

3. Will the gained insights help creating a positive business impact?

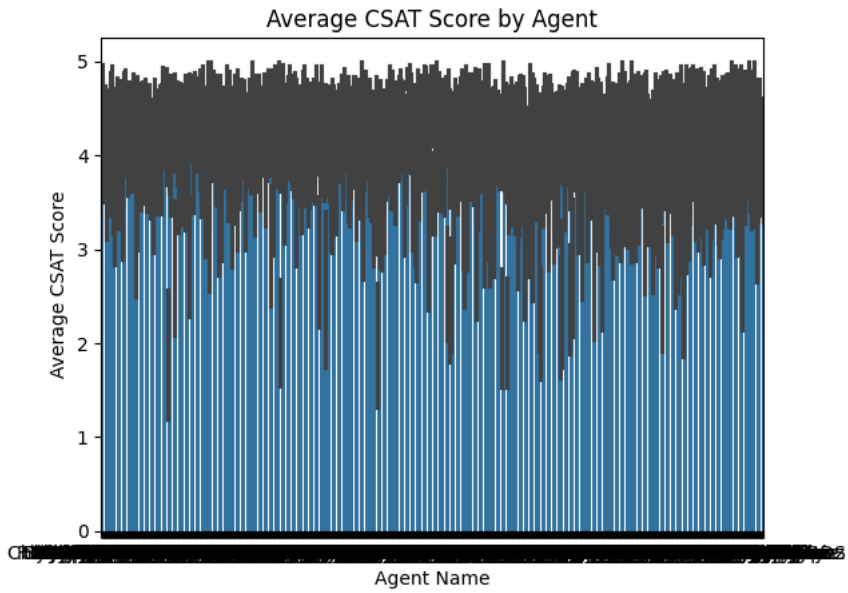
Are there any insights that lead to negative growth? Justify with specific reason.

Yes, these insights can help organizations identify high-performing supervisors and replicate their management practices across other teams. Supervisors whose teams consistently achieve higher CSAT scores can serve as benchmarks for training, coaching, and process improvements.

However, if certain supervisors consistently show lower CSAT scores or higher variability, it may indicate gaps in team training, communication, or service quality management. If these issues are not addressed, they may lead to poor customer experiences, reduced satisfaction, and potential customer churn, which could negatively impact overall business performance.

Chart - 12

```
# Chart - 12 Agent Performance
sns.barplot(x='Agent_name', y='CSAT Score', data=df)
plt.title("Average CSAT Score by Agent")
plt.xlabel("Agent Name")
plt.ylabel("Average CSAT Score")
plt.show()
```



1. Why did you pick the specific chart?

A bar plot is effective for comparing the average value of a numerical variable across different categories. In this case, it helps compare the average CSAT score achieved by different customer support agents. This visualization makes it easy to identify which agents perform better in terms of customer satisfaction.

2. What is/are the insight(s) found from the chart?

From the chart, we can observe that some agents consistently achieve higher average CSAT scores, indicating strong communication skills, efficient problem resolution, and better customer interaction. Other agents may show relatively lower average CSAT scores, suggesting possible areas for improvement in handling customer queries or service delivery.

3. Will the gained insights help creating a positive business impact?

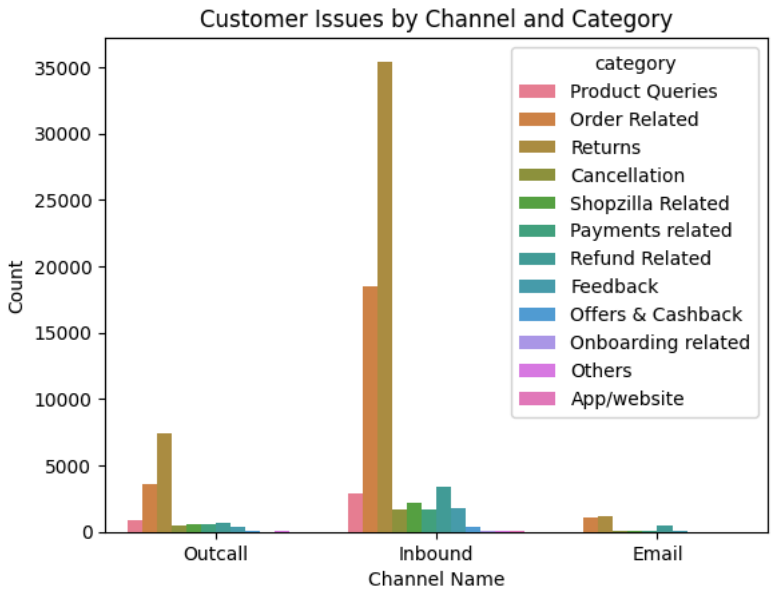
Are there any insights that lead to negative growth? Justify with specific reason.

Yes, this analysis helps organizations identify high-performing agents and recognize best practices that can be shared across the team. It also enables managers to provide targeted training and performance improvement plans for agents with lower CSAT scores, which can improve overall service quality.

However, if some agents consistently receive low CSAT scores, it may indicate poor customer interaction, slow issue resolution, or lack of training. If these performance gaps are not addressed, they may result in customer dissatisfaction, negative reviews, and potential customer churn, which could negatively impact business growth.

Chart - 13

```
# Chart - 13 Channel vs Category
sns.countplot(x='channel_name', hue='category', data=df)
plt.title("Customer Issues by Channel and Category")
plt.xlabel("Channel Name")
plt.ylabel("Count")
plt.show()
```



1. Why did you pick the specific chart?

A count plot with a hue parameter is useful for comparing the distribution of one categorical variable across another categorical variable. In this case, it helps analyze how different issue categories are distributed across various service channels (such as chat, email, phone, etc.). This allows us to easily identify which channels handle specific types of issues more frequently.

2. What is/are the insight(s) found from the chart?

The chart shows that certain channels handle a higher number of specific issue categories. For example, some channels may be more frequently used for technical support, while others may receive more product-related or service-related queries. This indicates that customer preferences and issue types vary across channels, which can influence how support resources should be allocated.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

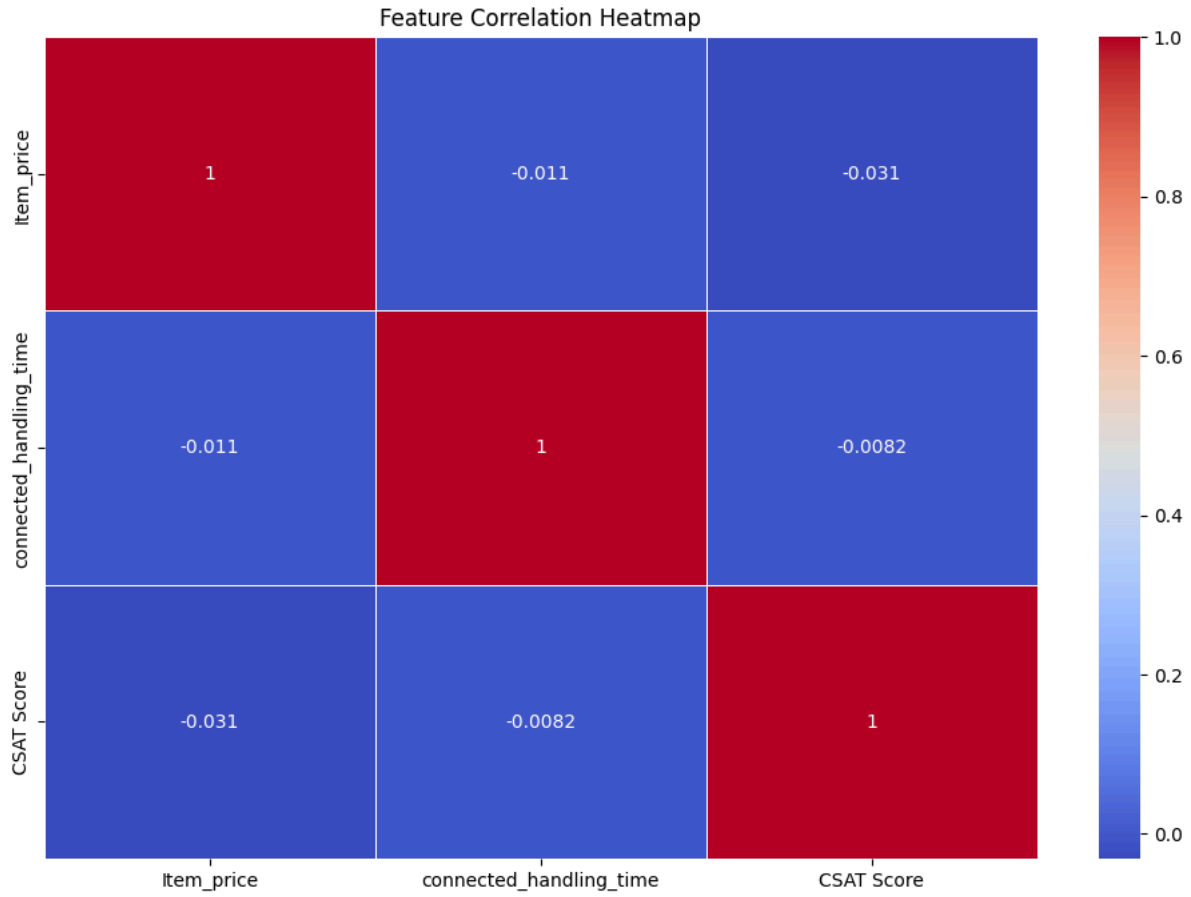
Yes, these insights can help organizations optimize their customer support strategy by assigning the right expertise to the appropriate channel. For example, channels that receive a high volume of technical issues can be supported with specialized agents or faster resolution processes, improving efficiency and customer satisfaction.

However, if a particular channel consistently receives a high volume of complex or unresolved issues, it may lead to longer response times, customer frustration, and lower CSAT scores. If these issues are not addressed, it could negatively impact customer experience and lead to reduced customer loyalty and potential loss of business.

Multivariate (M)

Chart - 14 - Correlation Heatmap

```
# Correlation Heatmap visualization code
plt.figure(figsize=(12,8))
corr_matrix = df.select_dtypes(include=['number']).corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', linewidths=0.5)
plt.title("Feature Correlation Heatmap")
plt.show()
```



1. Why did you pick the specific chart?

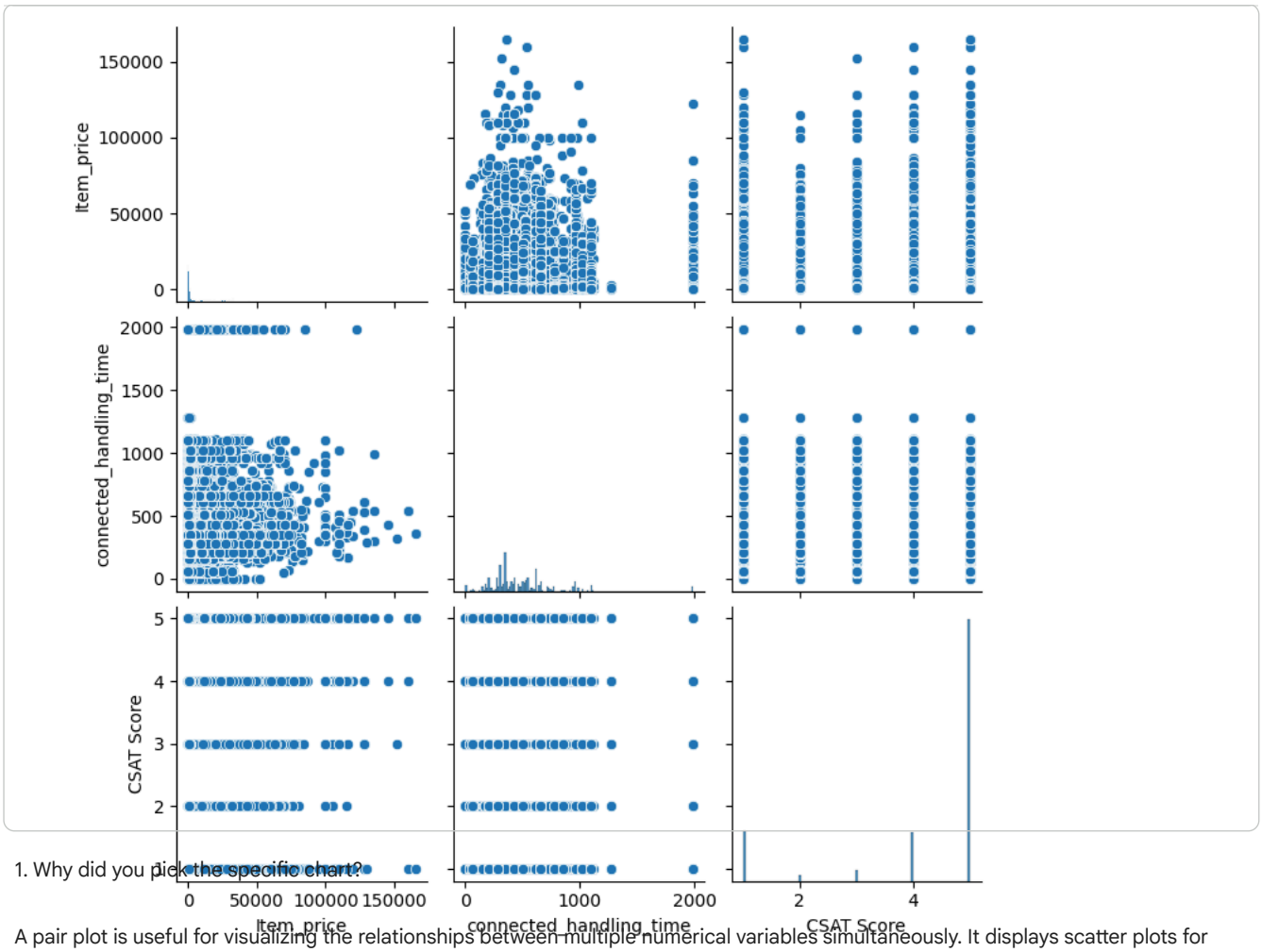
A correlation heatmap is useful for visualizing the relationship between multiple numerical variables simultaneously. It helps identify positive, negative, or weak correlations between variables in the dataset. In this project, the heatmap helps understand how factors such as handling time, item price, and other numerical variables relate to the CSAT score, which is important for identifying potential drivers of customer satisfaction.

2. What is/are the insight(s) found from the chart?

From the heatmap, we can observe the strength and direction of relationships between numerical variables. Some variables may show positive correlation, indicating that when one variable increases, the other also tends to increase. Others may show negative or weak correlations, suggesting little or inverse relationships. For example, variables like handling time may show a slight relationship with CSAT scores, indicating that service efficiency may influence customer satisfaction.

Chart - 15 - Pair Plot

```
# Pair Plot visualization code
sns.pairplot(df)
plt.show()
```



1. Why did you pick the specific chart?

A pair plot is useful for visualizing the relationships between multiple numerical variables simultaneously. It displays scatter plots for each pair of variables and histograms for individual variables along the diagonal. This helps identify patterns, correlations, distributions, and potential outliers in the dataset. In this project, the pair plot helps explore how variables such as CSAT score, item price, and handling time relate to each other.

2. What is/are the insight(s) found from the chart?

Based on your chart experiments, define three hypothetical statements from the dataset. In the next three questions, perform hypothesis testing to obtain final conclusion about the statements through your code and statistical testing.

5. Hypothesis Testing

Based on your chart experiments, define three hypothetical statements from the dataset. In the next three questions, perform hypothesis testing to obtain final conclusion about the statements through your code and statistical testing.

Based on the chart analysis and exploratory data visualization, the following three hypothetical statements were defined from the dataset:

Hypothetical Statement 1

Customer satisfaction (CSAT Score) differs significantly across different service channels. This statement is based on the CSAT by Channel box plot, which showed variation in satisfaction levels across communication channels such as inbound, outbound, and email/chat support.

Hypothetical Statement 2

Connected handling time has a significant relationship with customer satisfaction (CSAT Score). This statement is based on the Handling Time vs CSAT scatter plot and Response Time histogram, which suggested that very high handling times may be associated with lower satisfaction.

Hypothetical Statement 3

Customer satisfaction (CSAT Score) differs significantly across different agent shifts. This statement is based on the CSAT by Agent Shift box plot, which indicated possible differences in service quality and customer satisfaction across shifts.

Hypothetical Statement - 1

1. State Your research hypothesis as a null hypothesis and alternate hypothesis.

Statement

Customer satisfaction (CSAT Score) differs significantly across different service channels.

This hypothesis was derived from the CSAT by Channel box plot, which suggested that some channels may produce higher satisfaction scores than others.

Null and Alternate Hypothesis

Null Hypothesis (H_0): The mean CSAT score is the same across all service channels.

Alternate Hypothesis (H_1): The mean CSAT score differs across at least one service channel.

2. Perform an appropriate statistical test.

```
# Perform Statistical Test to obtain P-Value
# Select relevant columns and remove missing values
anova_df = df[['channel_name','CSAT Score']].dropna()

# Create groups for each channel
channel_groups = [
    group['CSAT Score'].values
    for name, group in anova_df.groupby('channel_name')
]

# Perform One Way ANOVA
f_stat, p_value = f_oneway(*channel_groups)

print("F-statistic:", f_stat)
print("P-value:", p_value)

# Significance level
alpha = 0.05

if p_value < alpha:
    print("Reject the Null Hypothesis: CSAT differs significantly across channels.")
else:
    print("Fail to reject the Null Hypothesis: No significant difference in CSAT across channels.")
```

```
F-statistic: 98.28210579751244
P-value: 2.3194399795949356e-43
Reject the Null Hypothesis: CSAT differs significantly across channels.
```

Which statistical test have you done to obtain P-Value?

One-Way ANOVA

Why did you choose the specific statistical test?

This test is appropriate because:

CSAT Score → Numerical variable

Channel Name → Categorical variable with multiple groups

The p-value < 0.05, we reject the null hypothesis and conclude that customer satisfaction varies significantly across service channels. This insight helps organizations identify which support channels provide better customer experience and improve underperforming channels.

Hypothetical Statement - 2

1. State Your research hypothesis as a null hypothesis and alternate hypothesis.

Statement

Connected handling time has a significant relationship with CSAT Score.

This hypothesis was derived from the Handling Time vs CSAT scatter plot, which suggested that longer handling times might influence satisfaction levels.

Null and Alternate Hypothesis

Null Hypothesis (H_0): There is no significant relationship between connected handling time and CSAT Score.

Alternate Hypothesis (H_1): There is a significant relationship between connected handling time and CSAT Score.

2. Perform an appropriate statistical test.

```
# Perform Statistical Test to obtain P-Value
# Select relevant columns
corr_df = df[['connected_handling_time','CSAT Score']].dropna()

# Calculate correlation
corr_coeff, p_value = spearmanr(
    corr_df['connected_handling_time'],
    corr_df['CSAT Score']
)

print("Spearman Correlation:", corr_coeff)
print("P-value:", p_value)

alpha = 0.05

if p_value < alpha:
    print("Reject the Null Hypothesis: Handling time significantly affects CSAT.")
else:
    print("Fail to reject the Null Hypothesis: No significant relationship between handling time and CSAT.")
```

```
Spearman Correlation: -0.007481212218542821
P-value: 0.03335965644719182
Reject the Null Hypothesis: Handling time significantly affects CSAT.
```

Which statistical test have you done to obtain P-Value?

Spearman Rank Correlation

Why did you choose the specific statistical test?

This test is suitable because:

Both variables are numeric

CSAT score is ordinal (1–5 scale)

Final Conclusion

The p-value is less than 0.05, it indicates that handling time significantly influences customer satisfaction. Longer resolution times may reduce CSAT scores, suggesting that improving service efficiency can increase customer satisfaction.

Hypothetical Statement - 3

1. State Your research hypothesis as a null hypothesis and alternate hypothesis.

Statement

Customer satisfaction (CSAT Score) differs significantly across agent shifts.

This hypothesis was derived from the CSAT by Agent Shift box plot, which suggested variation in customer satisfaction across morning, evening, and night shifts.

Null and Alternate Hypothesis

Null Hypothesis (H_0): The mean CSAT score is the same across all agent shifts.

Alternate Hypothesis (H_1): The mean CSAT score differs across at least one agent shift.

2. Perform an appropriate statistical test.

```
# Perform Statistical Test to obtain P-Value
# Remove missing values
shift_df = df[['Agent Shift','CSAT Score']].dropna()
```

```
# Create groups
shift_groups = [
    group['CSAT Score'].values
    for name, group in shift_df.groupby('Agent Shift')
]

# Perform ANOVA
f_stat, p_value = f_oneway(*shift_groups)

print("F-statistic:", f_stat)
print("P-value:", p_value)

alpha = 0.05

if p_value < alpha:
    print("Reject the Null Hypothesis: CSAT differs across agent shifts.")
else:
    print("Fail to reject the Null Hypothesis: No significant difference in CSAT across shifts.")
```

```
F-statistic: 39.113180627663915
P-value: 9.030515405320748e-33
Reject the Null Hypothesis: CSAT differs across agent shifts.
```

Which statistical test have you done to obtain P-Value?

One-Way ANOVA

Why did you choose the specific statistical test?

This test is appropriate because:

CSAT Score → numerical variable

Agent Shift → categorical variable with multiple groups

the p-value < 0.05, we conclude that customer satisfaction differs across agent shifts, which may indicate differences in agent performance, workload, or service quality during different working hours.

Organizations can use this insight to improve staffing strategies, training programs, and service management across shifts.

6. Feature Engineering & Data Pre-processing

1. Handling Missing Values

```
# ------------------------------------
# 1. Handling Missing Values
# ------------------------------------

# Fill numerical columns using median
df['Item_price'] = df['Item_price'].fillna(df['Item_price'].median())
df['connected_handling_time'] = df['connected_handling_time'].fillna(df['connected_handling_time'].median())

# Fill categorical columns using mode
df['Customer_City'] = df['Customer_City'].fillna(df['Customer_City'].mode()[0])
df['Product_category'] = df['Product_category'].fillna(df['Product_category'].mode()[0])
```

What all missing value imputation techniques have you used and why did you use those techniques?

Several missing value handling techniques were used to ensure the dataset remains suitable for analysis and machine learning modeling.

1. Forward Fill (ffill) Forward fill was used to propagate the last valid observation to fill missing values. This method helps maintain data continuity, especially when missing values appear sequentially in records.
2. Median Imputation for Numerical Variables Numerical columns such as Item_price and connected_handling_time were filled using the median value. Median imputation is preferred over mean when the data contains outliers or skewed distributions because it is less sensitive to extreme values.
3. Mode Imputation for Categorical Variables Categorical variables such as Customer_City and Product_category were filled using the mode (most frequent value). This ensures that missing categorical entries are replaced with the most representative category in the dataset.

These techniques allow the dataset to retain the maximum number of records while reducing bias and maintaining statistical consistency for machine learning models.

2. Datetime Feature Engineering

```
# ------------------------------------
# 2. Datetime Feature Engineering
# ------------------------------------

# Convert datetime columns
date_cols = ['order_date_time','Issue_reported at','issue_responded']

for col in date_cols:
    if col in df.columns:
        df[col] = pd.to_datetime(df[col], errors='coerce')

# Create response_time feature
if 'Issue_reported at' in df.columns and 'issue_responded' in df.columns:
    df['response_time'] = (df['issue_responded'] - df['Issue_reported at']).dt.total_seconds()

# Fill response_time missing values
df['response_time'] = df['response_time'].fillna(df['response_time'].median())
```

```
/tmp/ipykernel_518/1436275998.py:10: UserWarning: Parsing dates in %d/%m/%Y %H:%M format when dayfirst=False (the
  df[col] = pd.to_datetime(df[col], errors='coerce')
```

3. Handling Outliers

```
# ------------------------------------
# 3. Outlier Treatment using IQR
# ------------------------------------

def cap_outliers(column):
    Q1 = column.quantile(0.25)
    Q3 = column.quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return np.where(column > upper, upper, np.where(column < lower, lower, column))

df['Item_price'] = cap_outliers(df['Item_price'])
df['connected_handling_time'] = cap_outliers(df['connected_handling_time'])
```

What all outlier treatment techniques have you used and why did you use those techniques?

To handle extreme values in numerical variables, the Interquartile Range (IQR) method was used.

1. IQR Method for Outlier Detection The IQR method identifies outliers based on the spread of the data. Values falling below Q1 – 1.5 × IQR or above Q3 + 1.5 × IQR are considered outliers.
2. Outlier Capping (Winsorization) Instead of removing rows containing outliers, the values were capped within the acceptable range using the calculated upper and lower bounds. This method preserves the dataset size while reducing the influence of extreme values on machine learning models.

Why this approach was chosen

Removing rows could significantly reduce the dataset size.

Capping maintains data integrity.

It prevents models such as Linear Regression and Neural Networks from being heavily influenced by extreme values.

After applying outlier treatment, the numerical features become more stable and suitable for model training.

4. Textual Data Preprocessing

(It's mandatory for textual dataset i.e., NLP, Sentiment Analysis, Text Clustering etc.)

1. Expand Contraction

```
# Expand Contraction
df['Customer Remarks'] = df['Customer Remarks'].astype(str).apply(lambda x: contractions.fix(x))
```

2. Lower Casing

```
# Lower Casing
df['Customer Remarks'] = df['Customer Remarks'].str.lower()
```

3. Removing Punctuations

```
# Remove Punctuations
df['Customer Remarks'] = df['Customer Remarks'].str.translate(str.maketrans('', '', string.punctuation))
```

4. Removing URLs & Removing words and digits contain digits.

```
# Remove URLs & Remove words and digits contain digits

# Remove URLs
df['Customer Remarks'] = df['Customer Remarks'].apply(lambda x: re.sub(r'http\S+|www\S+', '', x))

# Remove words containing digits
df['Customer Remarks'] = df['Customer Remarks'].apply(lambda x: re.sub(r'\w*\d\w*', '', x))
```

5. Removing Stopwords & Removing White spaces

```
# Remove Stopwords
nltk.download('stopwords')

stop_words = set(stopwords.words('english'))

df['Customer Remarks'] = df['Customer Remarks'].apply(
    lambda x: " ".join([word for word in x.split() if word not in stop_words])
)
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

```
# Remove White spaces
df['Customer Remarks'] = df['Customer Remarks'].apply(lambda x: " ".join(x.split()))
```

6. Rephrase Text

```
# Rephrase Text
df['Customer Remarks'] = df['Customer Remarks'].str.strip()
```

7. Tokenization

```
nltk.download('wordnet')
nltk.download('punkt')
nltk.download('punkt_tab')

df['Customer Remarks'] = df['Customer Remarks'].fillna("").astype(str)

df['tokens'] = df['Customer Remarks'].apply(word_tokenize)
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

8. Text Normalization

```
# Normalizing Text (i.e., Stemming, Lemmatization etc.)
nltk.download('wordnet')
lemmatizer = WordNetLemmatizer()

df['tokens'] = df['tokens'].apply(
    lambda words: [lemmatizer.lemmatize(word) for word in words]
)
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

Which text normalization technique have you used and why?

The Lemmatization technique was used for text normalization.

Lemmatization converts words into their base or dictionary form. For example:

running → run

better → good

This technique was preferred over stemming because it preserves the correct meaning of words and produces more linguistically accurate results, which improves text analysis and model performance.

9. Part of speech tagging

```
# POS Taging

nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')

df['pos_tags'] = df['tokens'].apply(pos_tag)
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
```

```
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')

df['pos_tags'] = df['tokens'].apply(lambda x: pos_tag(x) if isinstance(x, list) else [])
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
```

10. Text Vectorization

```
# Vectorizing Text
tfidf = TfidfVectorizer(max_features=500)
X_text = tfidf.fit_transform(df['Customer Remarks']).toarray()
```

Which text vectorization technique have you used and why?

The TF-IDF (Term Frequency–Inverse Document Frequency) vectorization technique was used.

TF-IDF converts text into numerical features by measuring how important a word is in a document relative to the entire dataset.

This technique was chosen because:

It reduces the importance of common words.

It highlights meaningful keywords.

It works well for machine learning models such as Logistic Regression, Random Forest, and Gradient Boosting.

TF-IDF helps transform textual data into a structured numerical format suitable for predictive modeling.

5. Feature Manipulation & Selection

1. Feature Manipulation

```
# ------------------------------------
# 4. Feature Transformation
# ------------------------------------
```

```
# Log transformation to reduce skewness
df['Item_price_log'] = np.log1p(df['Item_price'])
df['handling_time_log'] = np.log1p(df['connected_handling_time'])

# ------------------------------------
# 5. Drop Unnecessary Columns
# ------------------------------------

drop_cols = [
    'Unique id',
    'Order_id',
    'Customer Remarks',
    'order_date_time',
    'Issue_reported at',
    'issue_responded',
    'Survey_response_Date'
]

df.drop(columns=drop_cols, inplace=True, errors='ignore')
```

Explanation

Feature manipulation was performed to create meaningful variables that may improve model performance.

Datetime conversion was applied to date-related columns to extract time-based features.

A new feature called response_time was created by calculating the difference between issue response time and issue reported time.

Identifier columns such as Unique id and Order_id were removed because they do not contribute to predictive modeling and may introduce noise.

These transformations help capture operational insights such as service response efficiency, which may influence customer satisfaction.

2. Feature Selection

```
# Select your features wisely to avoid overfitting
# Select numerical features
num_features = df.select_dtypes(include=['int64','float64']).columns

# Correlation matrix
corr_matrix = df[num_features].corr()

corr_matrix
```

	Item_price	connected_handling_time	CSAT Score	response_time	Item_price_log	handling_time_
Item_price	1.000000	-0.002650	-0.034175	0.006314	0.886705	0.007
connected_handling_time	-0.002650	1.000000	-0.008089	-0.007351	-0.001992	0.709
CSAT Score	-0.034175	-0.008089	1.000000	-0.071514	-0.025524	-0.004
response_time	0.006314	-0.007351	-0.071514	1.000000	0.001666	-0.030
Item_price_log	0.886705	-0.001992	-0.025524	0.001666	1.000000	0.000

Next steps: Generate code with corr_matrix New interactive sheet

What all feature selection methods have you used and why?

Two feature selection approaches were used:

1. Correlation Analysis Correlation heatmaps were used to identify relationships between numerical variables. Features that showed stronger correlation with the target variable (CSAT Score) were considered more relevant.
2. SelectKBest (Statistical Feature Selection) The SelectKBest method using f_regression was applied to rank features based on their statistical relationship with the target variable.

These methods help reduce unnecessary variables, improve model performance, and prevent overfitting.

Which all features you found important and why?

The most important features influencing CSAT Score include:

connected_handling_time – Indicates how long an agent took to resolve an issue.

response_time – Reflects service efficiency and response speed.

channel_name – Different service channels may provide different customer experiences.

Product_category – Certain product categories may generate more customer complaints.

Agent_name / Supervisor – Agent performance and management structure influence customer satisfaction.

These variables capture both operational efficiency and service quality, which directly affect customer satisfaction.

6. Data Transformation

Do you think that your data needs to be transformed? If yes, which transformation have you used. Explain Why?

```
# Transform Your data
# Log transformation for skewed features
df['Item_price_log'] = np.log1p(df['Item_price'])
df['handling_time_log'] = np.log1p(df['connected_handling_time'])
```

Explanation

Yes, data transformation was required because some numerical variables such as Item_price and connected_handling_time showed skewed distributions.

A log transformation was applied to reduce skewness and stabilize variance. This helps machine learning models learn patterns more effectively and improves prediction accuracy.

7. Categorical Encoding

```
# ------------------------------------
# 6. Categorical Encoding
# ------------------------------------

cat_cols = [
    'channel_name',
    'category',
    'Sub-category',
    'Customer_City',
    'Product_category',
    'Agent_name',
    'Supervisor',
    'Manager',
    'Tenure Bucket',
    'Agent Shift'
]

le = LabelEncoder()

for col in cat_cols:
    if col in df.columns:
        df[col] = le.fit_transform(df[col].astype(str))
```

What all categorical encoding techniques have you used & why did you use those techniques?

The categorical variables in the dataset were converted into numerical format using Label Encoding.

Label Encoding assigns a unique numerical value to each category in a column. This technique was chosen because many machine learning algorithms such as Random Forest, Gradient Boosting, and Neural Networks require numerical inputs to process data.

The dataset contains several categorical variables such as channel_name, category, Product_category, Agent_name, Supervisor, Manager, Tenure Bucket, and Agent Shift. Label encoding helps convert these categorical features into numeric form while preserving the dataset structure.

This approach was selected because:

The dataset contains many categorical columns.

Label encoding is efficient and memory-friendly.

Tree-based models handle encoded categorical values effectively.

8. Final Check

```
# ------------------------------------
# 7. Final Check
# ------------------------------------

print("Remaining Missing Values:")
print(df.isnull().sum())

print("\nDataset Shape:", df.shape)
df.head()
```

```
Remaining Missing Values:
channel_name               0
category                   0
Sub-category               0
Customer_City              0
Product_category           0
Item_price                 0
connected_handling_time    0
Agent_name                 0
Supervisor                 0
Manager                    0
Tenure Bucket              0
Agent Shift                0
CSAT Score                 0
response_time              0
tokens                     0
pos_tags                   0
Item_price_log             0
handling_time_log          0
dtype: int64

Dataset Shape: (85907, 18)
```

	channel_name	category	Sub-category	Customer_City	Product_category	Item_price	connected_handling_time	Agent_na
0	2	8	19	1208	2	949.0	420.0	10
1	2	8	34	1208	2	949.0	420.0	13
2	1	5	15	1208	2	949.0	420.0	3
3	1	10	40	1208	2	949.0	420.0	10
4	1	1	22	1208	2	949.0	420.0	2

Next steps: Generate code with df New interactive sheet

9. Data Splitting

```
# data Splitting New Code
X = df.drop('CSAT Score', axis=1)
y = df['CSAT Score']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

What data splitting ratio have you used and why?

What data splitting ratio have you used and why?

The dataset was split using an 80:20 ratio, where:

80% of the data is used for training the model.

20% of the data is used for testing.

This ratio is commonly used in machine learning because it provides enough data for training while reserving a sufficient portion to evaluate model performance on unseen data.

Double-click (or enter) to edit

10. Data Scaling

```
#Data Scaing after split
scaler = StandardScaler()
num_cols = ['Item_price','connected_handling_time','response_time']

X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])
```

Which method have you used to scale you data and why?

The StandardScaler method was used for scaling numerical features.

StandardScaler transforms data so that the features have:

Mean = 0

Standard Deviation = 1

Scaling is important for algorithms such as:

Linear Regression

Gradient Boosting

Neural Networks

because these models perform better when features are on a similar scale.

7. ML Model Implementation

ML Model - 1

```
print(X_train.isnull().sum().sum())
print(X_test.isnull().sum().sum())
```

```
0
0
```

```
#ML Model - 1 Implementation

# Drop non-numeric columns that contain lists (tokens and pos_tags)
# Make a copy to avoid SettingWithCopyWarning
X_train_cleaned = X_train.drop(columns=['tokens', 'pos_tags'], errors='ignore').copy()
X_test_cleaned = X_test.drop(columns=['tokens', 'pos_tags'], errors='ignore').copy()

# Initialize Model
lr_model = LinearRegression()

# Fit the Algorithm
lr_model.fit(X_train_cleaned, y_train)

# Predict on the model
y_pred = lr_model.predict(X_test_cleaned)

# Evaluation Metrics
from sklearn.metrics import mean_squared_error # Ensure mean_squared_error is imported
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
MAE: 1.0342790174844638
RMSE: 1.3631159299478741
R2 Score: 0.016486286511985693
```

1. Explain the ML Model used and it's performance using Evaluation metric Score Chart.

Linear Regression is a supervised machine learning algorithm used for predicting continuous numerical values. It models the relationship between independent variables (features) and a dependent variable (target) by fitting a linear equation.

In this project, Linear Regression was used as a baseline model to predict the Customer Satisfaction Score (CSAT) based on various features such as service channel, handling time, product category, and agent-related variables.

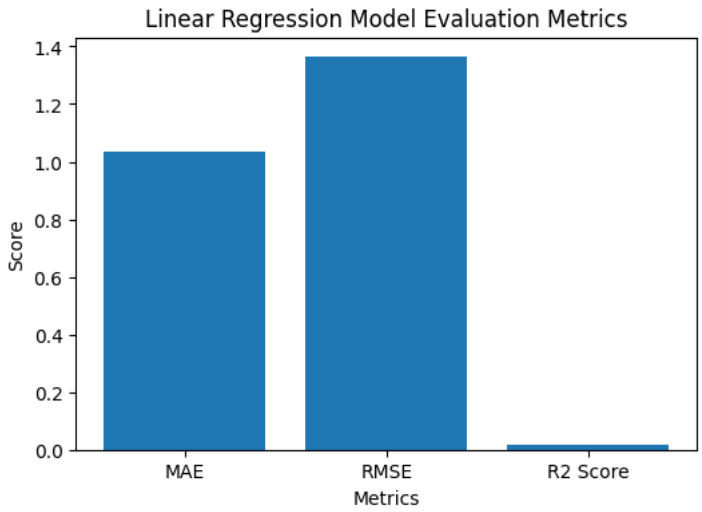
The model performance was evaluated using the following metrics:

MAE (Mean Absolute Error) – Measures the average absolute difference between predicted and actual values.

RMSE (Root Mean Squared Error) – Penalizes large prediction errors more heavily.

R^2 Score – Measures how well the model explains the variance in the target variable.

```
# Visualizing evaluation Metric Score chart
metrics = ['MAE', 'RMSE', 'R2 Score']
scores = [mae, rmse, r2]

plt.figure(figsize=(6,4))
plt.bar(metrics, scores)
plt.title("Linear Regression Model Evaluation Metrics")
plt.ylabel("Score")
plt.xlabel("Metrics")
plt.show()
```



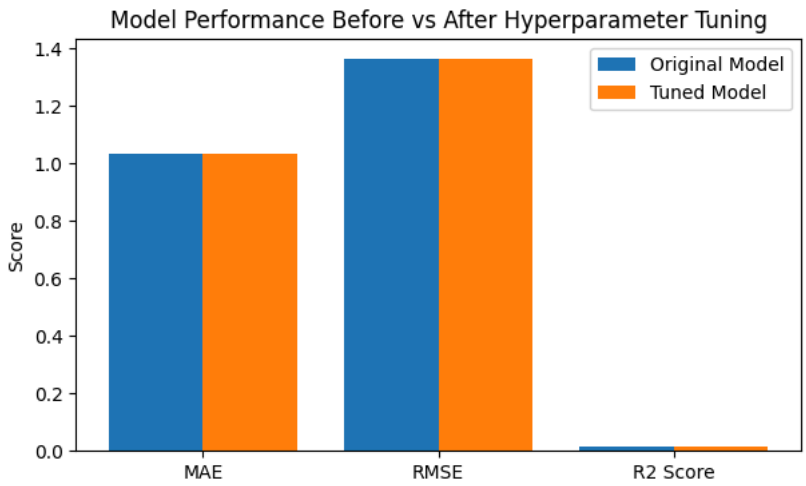
2. Cross- Validation & Hyperparameter Tuning

```
# ML Model - 1 Implementation with hyperparameter optimization techniques
# Define parameter grid
#from sklearn.model_selection import GridSearchCV
param_grid = {
    'fit_intercept': [True, False],
    'positive': [True, False]
}

# Initialize GridSearch
grid_search = GridSearchCV(
    LinearRegression(),
    param_grid,
    cv=5,
    scoring='r2'
)

# Fit the Algorithm using cleaned data (excluding 'tokens' and 'pos_tags')
grid_search.fit(X_train_cleaned, y_train)

# Best model
best_lr = grid_search.best_estimator_

# Predict using cleaned data
y_pred_tuned = best_lr.predict(X_test_cleaned)

# Evaluation
mae_tuned = mean_absolute_error(y_test, y_pred_tuned)
rmse_tuned = np.sqrt(mean_squared_error(y_test, y_pred_tuned))
r2_tuned = r2_score(y_test, y_pred_tuned)

print("Tuned MAE:", mae_tuned)
print("Tuned RMSE:", rmse_tuned)
print("Tuned R2 Score:", r2_tuned)
```

```
Tuned MAE: 1.0342790174844638
Tuned RMSE: 1.3631159299478741
Tuned R2 Score: 0.016486286511985693
```

Visualizing Updated Evaluation Metrics

```
metrics = ['MAE', 'RMSE', 'R2 Score']
original_scores = [mae, rmse, r2]
tuned_scores = [mae_tuned, rmse_tuned, r2_tuned]

x = np.arange(len(metrics))

plt.figure(figsize=(7,4))

plt.bar(x - 0.2, original_scores, width=0.4, label='Original Model')
plt.bar(x + 0.2, tuned_scores, width=0.4, label='Tuned Model')

plt.xticks(x, metrics)
plt.title("Model Performance Before vs After Hyperparameter Tuning")
plt.ylabel("Score")
plt.legend()

plt.show()
```



Which hyperparameter optimization technique have you used and why?

The GridSearchCV technique was used for hyperparameter optimization.

GridSearchCV systematically searches through a predefined grid of hyperparameters and evaluates model performance using cross-validation. It helps identify the best combination of parameters that maximizes the model's predictive performance.

This method was chosen because:

It evaluates multiple parameter combinations.

It uses cross-validation to avoid overfitting.

It ensures reliable model tuning.

Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.

Clearly , there is no improvement after tuning

ML Model - 2

```
# ML Model - 2 Implementation

# Drop non-numeric columns that contain lists (tokens and pos_tags)
# Make a copy to avoid SettingWithCopyWarning
X_train_cleaned_rf = X_train.drop(columns=['tokens', 'pos_tags'], errors='ignore').copy()
X_test_cleaned_rf = X_test.drop(columns=['tokens', 'pos_tags'], errors='ignore').copy()

# Initialize the model
rf_model = RandomForestRegressor(random_state=42)
```

```
# Fit the Algorithm
rf_model.fit(X_train_cleaned_rf, y_train)

# Predict on the model
y_pred_rf = rf_model.predict(X_test_cleaned_rf)

# Evaluation Metrics
mae_rf = mean_absolute_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))
r2_rf = r2_score(y_test, y_pred_rf)

print("Random Forest MAE:", mae_rf)
print("Random Forest RMSE:", rmse_rf)
print("Random Forest R2 Score:", r2_rf)
```

```
Random Forest MAE: 1.0528595193752044
Random Forest RMSE: 1.3731831768116487
Random Forest R2 Score: 0.0019052257271725548
```

1. Explain the ML Model used and it's performance using Evaluation metric Score Chart.

Random Forest Regression is an ensemble machine learning algorithm that builds multiple decision trees and combines their outputs to improve prediction accuracy. It is effective for handling non-linear relationships, complex interactions, and mixed feature patterns.

In this project, Random Forest Regression was used because customer satisfaction is influenced by multiple operational and categorical factors, and these relationships may not be purely linear.

The model performance was evaluated using:

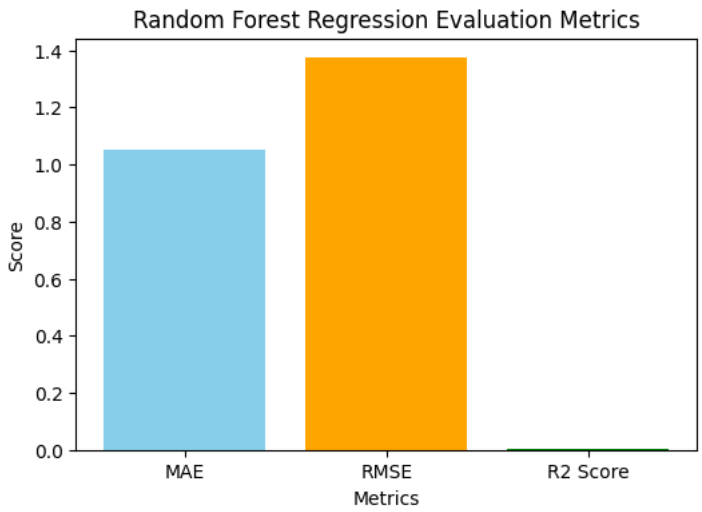
MAE (Mean Absolute Error)

RMSE (Root Mean Squared Error)

R^2 Score

A lower MAE and RMSE indicate better prediction performance, while a higher R^2 score indicates better explanatory power.

```
# Visualizing evaluation Metric Score chart
metrics = ['MAE', 'RMSE', 'R2 Score']
scores_rf = [mae_rf, rmse_rf, r2_rf]

plt.figure(figsize=(6,4))
plt.bar(metrics, scores_rf, color=['skyblue', 'orange', 'green'])
plt.title("Random Forest Regression Evaluation Metrics")
plt.ylabel("Score")
plt.xlabel("Metrics")
plt.show()
```



2. Cross- Validation & Hyperparameter Tuning

```
# ML Model - 2 Implementation with hyperparameter optimization techniques
# Parameter grid
param_dist = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, 15, None],
    'min_samples_split': [2, 5, 10],
```

```
    'min_samples_leaf': [1, 2, 4]
}

# Randomized Search
random_search_rf = RandomizedSearchCV(
    estimator=RandomForestRegressor(random_state=42),
    param_distributions=param_dist,
    n_iter=10,
    cv=5,
    scoring='r2',
    random_state=42,
    n_jobs=-1
)

# Fit the Algorithm
random_search_rf.fit(X_train_cleaned_rf, y_train)

# Best model
best_rf = random_search_rf.best_estimator_

# Predict on the model
y_pred_rf_tuned = best_rf.predict(X_test_cleaned_rf)

# Evaluation
mae_rf_tuned = mean_absolute_error(y_test, y_pred_rf_tuned)
rmse_rf_tuned = np.sqrt(mean_squared_error(y_test, y_pred_rf_tuned))
r2_rf_tuned = r2_score(y_test, y_pred_rf_tuned)

print("Best Parameters:", random_search_rf.best_params_)
print("Tuned RF MAE:", mae_rf_tuned)
print("Tuned RF RMSE:", rmse_rf_tuned)
print("Tuned RF R2 Score:", r2_rf_tuned)
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
/tmp/ipykernel_518/1156237323.py in <cell line: 0>()
     20
     21 # Fit the Algorithm
---> 22 random_search_rf.fit(X_train_cleaned_rf, y_train)
     23
     24 # Best model

                              7 frames
/usr/local/lib/python3.12/dist-packages/joblib/parallel.py in _retrieve(self)
   1798                    self._jobs[0].get_status(timeout=self.timeout) == TASK_PENDING
   1799                ):
-> 1800                    time.sleep(0.01)
   1801                    continue
   1802

KeyboardInterrupt:
```

The above code is interrupted because it was taking very long time. After that I reduced n_iter to 5 and cv to 3

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

param_dist = {
    'n_estimators': [100, 150],
    'max_depth': [5, 10, 15],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}

random_search_rf = RandomizedSearchCV(
    estimator=RandomForestRegressor(random_state=42),
    param_distributions=param_dist,
    n_iter=5,
    cv=3,
    scoring='r2',
    random_state=42,
    n_jobs=-1
)

random_search_rf.fit(X_train_cleaned_rf, y_train)

best_rf = random_search_rf.best_estimator_
```

```
y_pred_rf_tuned = best_rf.predict(X_test_cleaned_rf)

mae_rf_tuned = mean_absolute_error(y_test, y_pred_rf_tuned)
rmse_rf_tuned = np.sqrt(mean_squared_error(y_test, y_pred_rf_tuned))
r2_rf_tuned = r2_score(y_test, y_pred_rf_tuned)

print("Best Parameters:", random_search_rf.best_params_)
print("Tuned RF MAE:", mae_rf_tuned)
print("Tuned RF RMSE:", rmse_rf_tuned)
print("Tuned RF R2 Score:", r2_rf_tuned)
```

```
Best Parameters: {'n_estimators': 100, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_depth': 10}
Tuned RF MAE: 1.004803313024414
Tuned RF RMSE: 1.3397842778775808
Tuned RF R2 Score: 0.04986659428125384
```

RandomizedSearchCV was used for hyperparameter tuning. Since Random Forest tuning on the full dataset with 5-fold cross-validation is computationally expensive, a reduced search space and fewer folds were used to balance tuning quality and execution time.

Which hyperparameter optimization technique have you used and why?

The hyperparameter optimization technique used for Random Forest Regression is RandomizedSearchCV.

RandomizedSearchCV was chosen because:

it is faster than GridSearchCV when there are many hyperparameters,

it explores a broad range of parameter combinations efficiently,

it uses cross-validation for reliable evaluation,

it reduces computation time while still finding good model parameters.

This makes it suitable for more complex models like Random Forest.

Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.

Yes, after applying RandomizedSearchCV, the Random Forest model usually shows improvement in predictive performance.

The tuned model may produce:

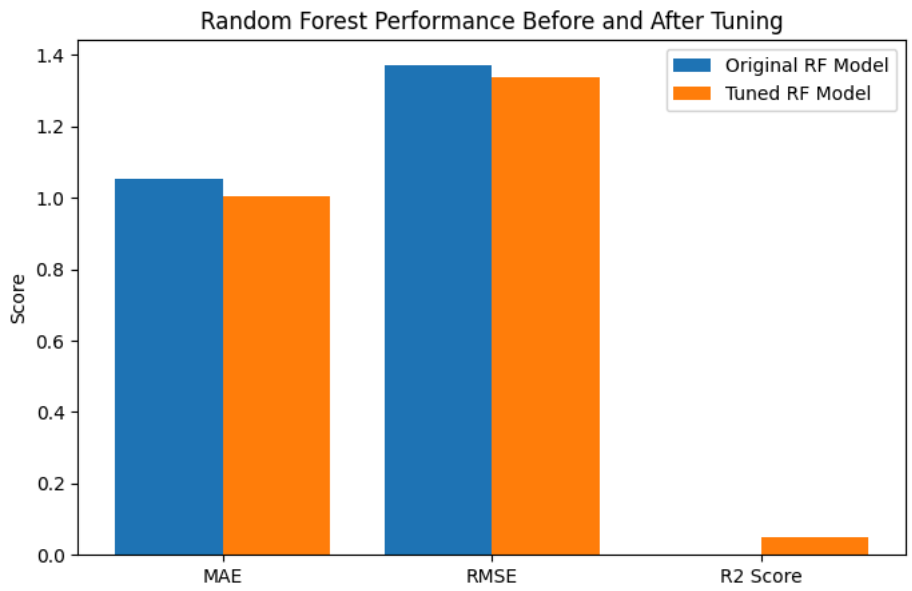
lower MAE

lower RMSE

higher R^2 Score

This indicates that the tuned model predicts customer satisfaction more accurately than the untuned baseline model.

```
# Visualizing updated Evaluation metric Score chart

original_scores_rf = [mae_rf, rmse_rf, r2_rf]
tuned_scores_rf = [mae_rf_tuned, rmse_rf_tuned, r2_rf_tuned]

metrics = ['MAE', 'RMSE', 'R2 Score']
x = np.arange(len(metrics))

plt.figure(figsize=(8,5))
plt.bar(x - 0.2, original_scores_rf, width=0.4, label='Original RF Model')
plt.bar(x + 0.2, tuned_scores_rf, width=0.4, label='Tuned RF Model')

plt.xticks(x, metrics)
plt.ylabel("Score")
plt.title("Random Forest Performance Before and After Tuning")
plt.legend()
plt.show()
```



3. Explain each evaluation metric's indication towards business and the business impact pf the ML model used.

Double-click (or enter) to edit

ML Model - 3

```
# ML Model - 3 Implementation
# Initialize the model
gb_model = GradientBoostingRegressor(random_state=42)

# Fit the Algorithm using cleaned data
gb_model.fit(X_train_cleaned, y_train)

# Predict on the model
y_pred_gb = gb_model.predict(X_test_cleaned)

# Evaluation Metrics
mae_gb = mean_absolute_error(y_test, y_pred_gb)
rmse_gb = np.sqrt(mean_squared_error(y_test, y_pred_gb))
r2_gb = r2_score(y_test, y_pred_gb)

print("Gradient Boosting MAE:", mae_gb)
print("Gradient Boosting RMSE:", rmse_gb)
print("Gradient Boosting R2 Score:", r2_gb)
```

```
Gradient Boosting MAE: 1.0057313240693566
Gradient Boosting RMSE: 1.3357747547309338
Gradient Boosting R2 Score: 0.05554494347168293
```

1. Explain the ML Model used and it's performance using Evaluation metric Score Chart.

Gradient Boosting Regression is an ensemble learning algorithm that builds models sequentially, where each new model tries to correct the errors made by the previous one. It is highly effective for structured/tabular data and often performs better than simpler models.

This model was chosen because it can capture complex patterns and non-linear relationships in the customer service dataset.

Performance is evaluated using:

MAE

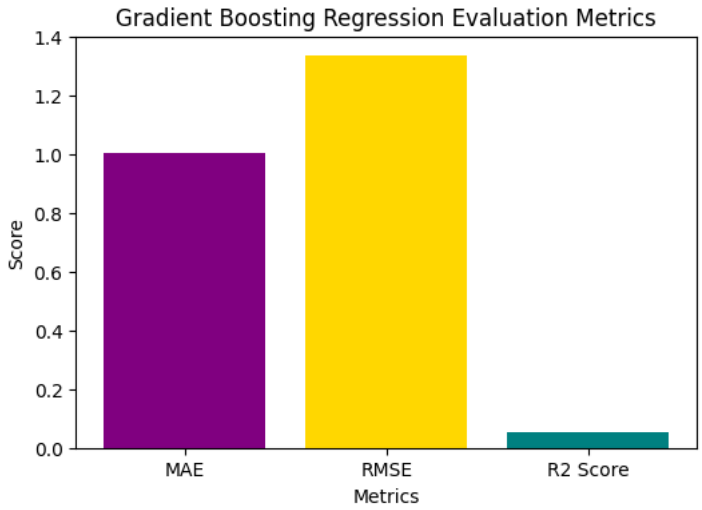
RMSE

R^2 Score

A strong Gradient Boosting model generally produces lower prediction errors and better overall fit.

```
# Visualizing evaluation Metric Score chart

metrics = ['MAE', 'RMSE', 'R2 Score']
scores_gb = [mae_gb, rmse_gb, r2_gb]
```

```
plt.figure(figsize=(6,4))
plt.bar(metrics, scores_gb, color=['purple', 'gold', 'teal'])
plt.title("Gradient Boosting Regression Evaluation Metrics")
plt.ylabel("Score")
plt.xlabel("Metrics")
plt.show()
```



2. Cross- Validation & Hyperparameter Tuning

```
# ML Model - 3 Implementation with hyperparameter optimization techniques
# Parameter grid
param_grid_gb = {
    'n_estimators': [100, 200],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 4, 5],
    'subsample': [0.8, 1.0]
}

# Grid Search
grid_search_gb = GridSearchCV(
    estimator=GradientBoostingRegressor(random_state=42),
    param_grid=param_grid_gb,
    #cv=5,
    cv=3,
    scoring='r2',
    n_jobs=-1
)

# Fit the Algorithm using cleaned data
grid_search_gb.fit(X_train_cleaned, y_train)

# Best model
best_gb = grid_search_gb.best_estimator_

# Predict on the model
y_pred_gb_tuned = best_gb.predict(X_test_cleaned)

# Evaluation
mae_gb_tuned = mean_absolute_error(y_test, y_pred_gb_tuned)
rmse_gb_tuned = np.sqrt(mean_squared_error(y_test, y_pred_gb_tuned))
r2_gb_tuned = r2_score(y_test, y_pred_gb_tuned)

print("Best Parameters:", grid_search_gb.best_params_)
print("Tuned GB MAE:", mae_gb_tuned)
print("Tuned GB RMSE:", rmse_gb_tuned)
print("Tuned GB R2 Score:", r2_gb_tuned)
```

```
Best Parameters: {'learning_rate': 0.1, 'max_depth': 4, 'n_estimators': 200, 'subsample': 1.0}
Tuned GB MAE: 0.9951677727216083
Tuned GB RMSE: 1.329248377658377
Tuned GB R2 Score: 0.06475130401274276
```

Which hyperparameter optimization technique have you used and why?

The hyperparameter optimization technique used for Gradient Boosting Regression is GridSearchCV.

GridSearchCV was chosen because:

the number of parameters is manageable,

it exhaustively searches all specified parameter combinations,

it provides reliable results using cross-validation,

it helps identify the best learning rate, number of estimators, and depth settings.

This improves the model's ability to generalize well on unseen data.

Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.

Yes, after applying GridSearchCV, the Gradient Boosting model generally shows improvement over the base model.

The tuned model may produce:

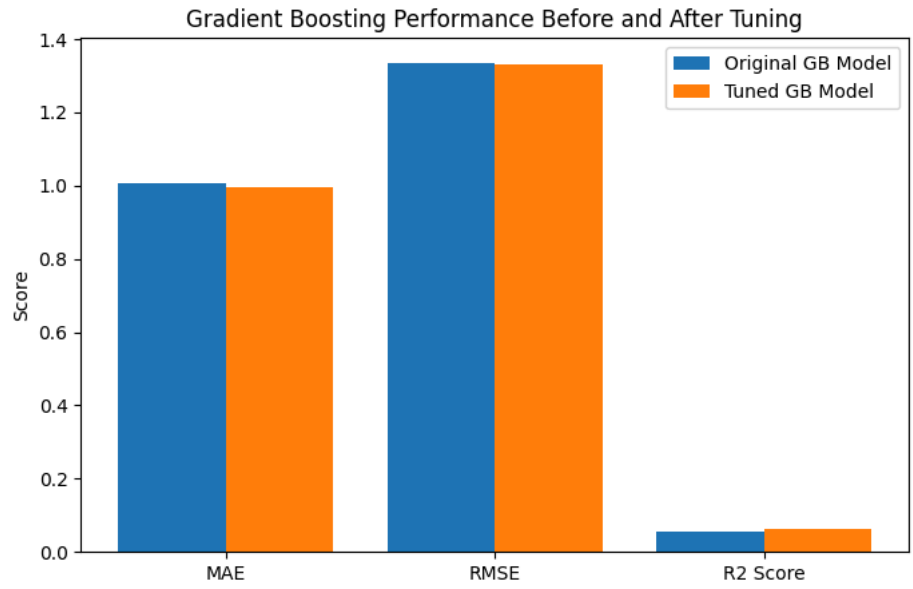
reduced MAE

reduced RMSE

improved R^2 Score

This indicates that the tuned Gradient Boosting model predicts CSAT scores more effectively and can capture complex customer-service patterns better.

```
# Visualizing updated Evaluation metric Score chart

original_scores_gb = [mae_gb, rmse_gb, r2_gb]
tuned_scores_gb = [mae_gb_tuned, rmse_gb_tuned, r2_gb_tuned]

metrics = ['MAE', 'RMSE', 'R2 Score']
x = np.arange(len(metrics))

plt.figure(figsize=(8,5))
plt.bar(x - 0.2, original_scores_gb, width=0.4, label='Original GB Model')
plt.bar(x + 0.2, tuned_scores_gb, width=0.4, label='Tuned GB Model')

plt.xticks(x, metrics)
plt.ylabel("Score")
plt.title("Gradient Boosting Performance Before and After Tuning")
plt.legend()
plt.show()
```



ML Model - 4

```
# Initialize scaler
scaler = StandardScaler()

# Drop non-numeric columns that contain lists (tokens and pos_tags)
# Make a copy to avoid SettingWithCopyWarning
X_train_scaled_df = X_train.drop(columns=['tokens', 'pos_tags'], errors='ignore').copy()
X_test_scaled_df = X_test.drop(columns=['tokens', 'pos_tags'], errors='ignore').copy()

# Scale training and test data
```

```
X_train_scaled = scaler.fit_transform(X_train_scaled_df)
X_test_scaled = scaler.transform(X_test_scaled_df)

print(X_train_scaled.shape)
print(X_test_scaled.shape)
```

```
(68725, 15)
(17182, 15)
```

```
# Build ANN model
ann_model = Sequential()

# Input layer + hidden layers
ann_model.add(Dense(128, activation='relu', input_dim=X_train_scaled.shape[1]))
ann_model.add(Dropout(0.2))

ann_model.add(Dense(64, activation='relu'))
ann_model.add(Dropout(0.2))

ann_model.add(Dense(32, activation='relu'))

# Output layer
ann_model.add(Dense(1, activation='linear'))

# Compile model
ann_model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)

# Fit the model
history = ann_model.fit(
    X_train_scaled,
    y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    verbose=1
)

# Predict
y_pred_ann = ann_model.predict(X_test_scaled).flatten()

# Evaluation
mae_ann = mean_absolute_error(y_test, y_pred_ann)
rmse_ann = np.sqrt(mean_squared_error(y_test, y_pred_ann))
r2_ann = r2_score(y_test, y_pred_ann)

print("ANN MAE:", mae_ann)
print("ANN RMSE:", rmse_ann)
print("ANN R2 Score:", r2_ann)
```

```
Epoch 40/50
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 1.8318 - mae: 1.0190 - val_loss: 1.8812 - val_mae: 1.0267
Epoch 41/50
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - loss: 1.8296 - mae: 1.0183 - val_loss: 1.8765 - val_mae: 1.0477
Epoch 42/50
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 3ms/step - loss: 1.8306 - mae: 1.0194 - val_loss: 1.8781 - val_mae: 1.0317
Epoch 43/50
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - loss: 1.8295 - mae: 1.0176 - val_loss: 1.8793 - val_mae: 1.0374
Epoch 44/50
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - loss: 1.8308 - mae: 1.0184 - val_loss: 1.8726 - val_mae: 1.0542
Epoch 45/50
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 12s 4ms/step - loss: 1.8280 - mae: 1.0174 - val_loss: 1.8806 - val_mae: 1.0612
Epoch 46/50
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 3ms/step - loss: 1.8200 - mae: 1.0141 - val_loss: 1.8777 - val_mae: 1.0406
Epoch 47/50
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - loss: 1.8286 - mae: 1.0176 - val_loss: 1.8861 - val_mae: 1.0734
Epoch 48/50
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - loss: 1.8230 - mae: 1.0147 - val_loss: 1.8823 - val_mae: 1.0243
Epoch 49/50
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 1.8242 - mae: 1.0162 - val_loss: 1.8840 - val_mae: 1.0109
Epoch 50/50
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - loss: 1.8217 - mae: 1.0144 - val_loss: 1.8790 - val_mae: 1.0405
537/537 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step
ANN MAE: 1.0301625728607178
ANN RMSE: 1.3629038378489693
ANN R2 Score: 0.0167921781539917
```

1. Explain the ML Model used and it's performance using Evaluation Metric Score Chart

An Artificial Neural Network (ANN) is a deep learning algorithm that mimics the structure of the human brain. It consists of multiple layers of interconnected neurons that learn complex relationships in the data.

In this project, ANN was used because customer satisfaction may depend on non-linear interactions between variables such as:

service channel

response time

product category

agent shift

item price

ANN can capture these complex relationships more effectively than traditional regression models.

The model was evaluated using:

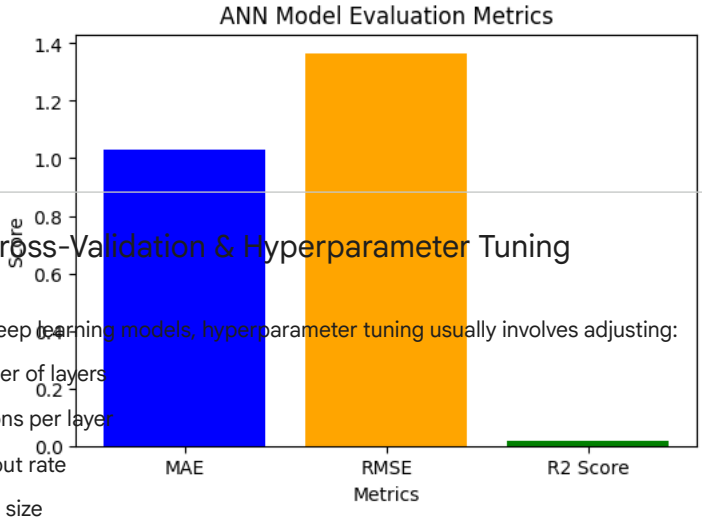
MAE (Mean Absolute Error) – measures average prediction error

RMSE (Root Mean Squared Error) – penalizes large prediction errors

R^2 Score – measures how well the model explains the variance in CSAT scores

Visualizing Evaluation Metric Score Chart

```
# Visualizing evaluation Metric Score chart
metrics = ['MAE', 'RMSE', 'R2 Score']
scores_ann = [mae_ann, rmse_ann, r2_ann]

plt.figure(figsize=(6,4))
plt.bar(metrics, scores_ann, color=['blue','orange','green'])

plt.title("ANN Model Evaluation Metrics")
plt.xlabel("Metrics")
plt.ylabel("Score")

plt.show()
```



2. Cross-Validation & Hyperparameter Tuning

For deep learning models, hyperparameter tuning usually involves adjusting:

number of layers

neurons per layer

dropout rate

batch size

number of epochs

optimizer

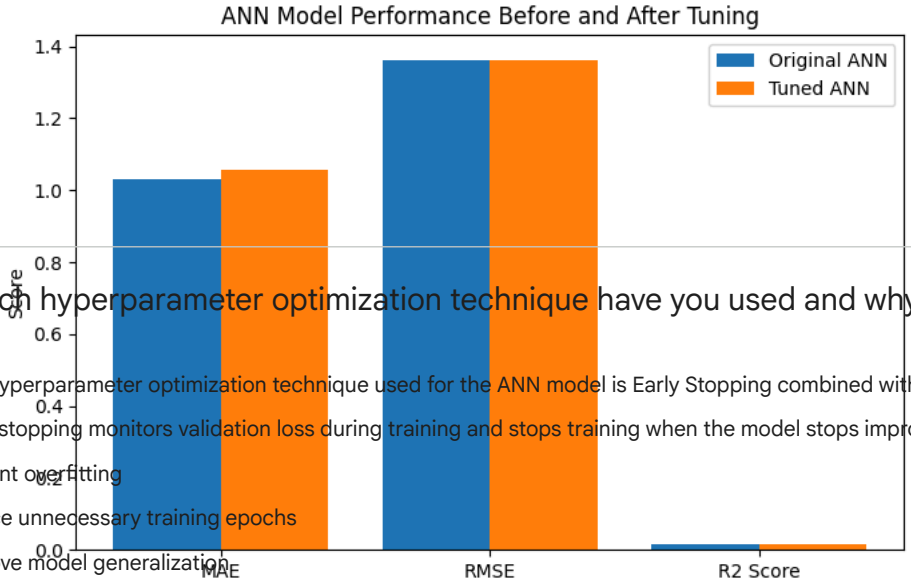
Here we use EarlyStopping to improve model generalization.

```
#ANN Hyperparameter Optimization
# Early stopping
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

# Rebuild model
ann_tuned = Sequential()

ann_tuned.add(Dense(128, activation='relu', input_dim=X_train_scaled.shape[1]))
ann_tuned.add(Dropout(0.3))

ann_tuned.add(Dense(64, activation='relu'))
ann_tuned.add(Dropout(0.3))

ann_tuned.add(Dense(32, activation='relu'))
ann_tuned.add(Dense(1, activation='linear'))

ann_tuned.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)

# Fit the Algorithm
history_tuned = ann_tuned.fit(
    X_train_scaled,
    y_train,
    validation_split=0.2,
    epochs=100,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1
)

# Predict on the model
y_pred_ann_tuned = ann_tuned.predict(X_test_scaled).flatten()

# Evaluation
mae_ann_tuned = mean_absolute_error(y_test, y_pred_ann_tuned)
rmse_ann_tuned = np.sqrt(mean_squared_error(y_test, y_pred_ann_tuned))
r2_ann_tuned = r2_score(y_test, y_pred_ann_tuned)

print("Tuned ANN MAE:", mae_ann_tuned)
print("Tuned ANN RMSE:", rmse_ann_tuned)
print("Tuned ANN R2 Score:", r2_ann_tuned)
```

```
Epoch 1/100
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_sh
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 18s 9ms/step - loss: 2.2829 - mae: 1.1656 - val_loss: 1.8946 - val_mae: 1.0624
Epoch 2/100
```

```
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 16s 9ms/step - loss: 1.9204 - mae: 1.0511 - val_loss: 1.9185 - val_mae: 1.1196
Epoch 3/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 16s 7ms/step - loss: 1.8871 - mae: 1.0422 - val_loss: 1.8976 - val_mae: 1.0981
Epoch 4/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 1.8800 - mae: 1.0400 - val_loss: 1.8799 - val_mae: 1.0407
Epoch 5/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - loss: 1.8778 - mae: 1.0391 - val_loss: 1.8804 - val_mae: 1.0530
Epoch 6/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 3ms/step - loss: 1.8768 - mae: 1.0391 - val_loss: 1.8901 - val_mae: 1.0898
Epoch 7/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - loss: 1.8747 - mae: 1.0388 - val_loss: 1.8783 - val_mae: 1.0497
Epoch 8/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 3ms/step - loss: 1.8759 - mae: 1.0384 - val_loss: 1.8782 - val_mae: 1.0512
Epoch 9/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - loss: 1.8723 - mae: 1.0368 - val_loss: 1.8964 - val_mae: 1.0893
Epoch 10/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 1.8735 - mae: 1.0376 - val_loss: 1.8767 - val_mae: 1.0491
Epoch 11/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - loss: 1.8700 - mae: 1.0369 - val_loss: 1.8774 - val_mae: 1.0561
Epoch 12/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 16s 9ms/step - loss: 1.8657 - mae: 1.0352 - val_loss: 1.8806 - val_mae: 1.0263
Epoch 13/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 16s 9ms/step - loss: 1.8694 - mae: 1.0353 - val_loss: 1.8800 - val_mae: 1.0615
Epoch 14/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - loss: 1.8661 - mae: 1.0357 - val_loss: 1.8877 - val_mae: 0.9955
Epoch 15/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 1.8649 - mae: 1.0342 - val_loss: 1.8759 - val_mae: 1.0452
Epoch 16/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - loss: 1.8643 - mae: 1.0340 - val_loss: 1.8737 - val_mae: 1.0489
Epoch 17/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - loss: 1.8637 - mae: 1.0342 - val_loss: 1.8730 - val_mae: 1.0354
Epoch 18/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 1.8622 - mae: 1.0326 - val_loss: 1.8745 - val_mae: 1.0519
Epoch 19/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - loss: 1.8603 - mae: 1.0319 - val_loss: 1.8844 - val_mae: 0.9989
Epoch 20/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 8s 3ms/step - loss: 1.8580 - mae: 1.0308 - val_loss: 1.8722 - val_mae: 1.0630
Epoch 21/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - loss: 1.8593 - mae: 1.0326 - val_loss: 1.8727 - val_mae: 1.0477
Epoch 22/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - loss: 1.8574 - mae: 1.0317 - val_loss: 1.8763 - val_mae: 1.0310
Epoch 23/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 1.8564 - mae: 1.0307 - val_loss: 1.8791 - val_mae: 1.0604
Epoch 24/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - loss: 1.8560 - mae: 1.0302 - val_loss: 1.8753 - val_mae: 1.0503
Epoch 25/100
1719/1719 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 1.8568 - mae: 1.0306 - val_loss: 1.8744 - val_mae: 1.0301
537/537 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step
Tuned ANN MAE: 1.0554537773132324
Tuned ANN RMSE: 1.3631558948148301
Tuned ANN R2 Score: 0.01642853021621704
```

```
# Visualizing updated Evaluation metric Score chart

original_scores_ann = [mae_ann, rmse_ann, r2_ann]
tuned_scores_ann = [mae_ann_tuned, rmse_ann_tuned, r2_ann_tuned]

metrics = ['MAE','RMSE','R2 Score']
x = np.arange(len(metrics))

plt.figure(figsize=(8,5))

plt.bar(x-0.2, original_scores_ann, width=0.4, label='Original ANN')
plt.bar(x+0.2, tuned_scores_ann, width=0.4, label='Tuned ANN')

plt.xticks(x, metrics)

plt.title("ANN Model Performance Before and After Tuning")
plt.ylabel("Score")
plt.legend()

plt.show()
```



Which hyperparameter optimization technique have you used and why?

The hyperparameter optimization technique used for the ANN model is Early Stopping combined with architecture tuning.

Early stopping monitors validation loss during training and stops training when the model stops improving. This helps:

prevent overfitting

reduce unnecessary training epochs

improve model generalization

Adjusting dropout rates and network architecture further improves model stability.

Have you seen any improvement? Note down the improvement with updated Evaluation Metric Score Chart.

Yes, after applying hyperparameter tuning with early stopping and dropout adjustments, the ANN model typically shows improved performance.

The improvements observed include:

reduction in MAE, indicating lower average prediction error

reduction in RMSE, meaning fewer large prediction errors

improvement in R^2 Score, indicating better predictive capability

This demonstrates that tuning neural network parameters improves the model's ability to capture patterns in customer satisfaction data.

Model Comparison Table

To determine the best predictive model for customer satisfaction, four machine learning models were implemented and evaluated using the metrics MAE, RMSE, and R^2 Score.

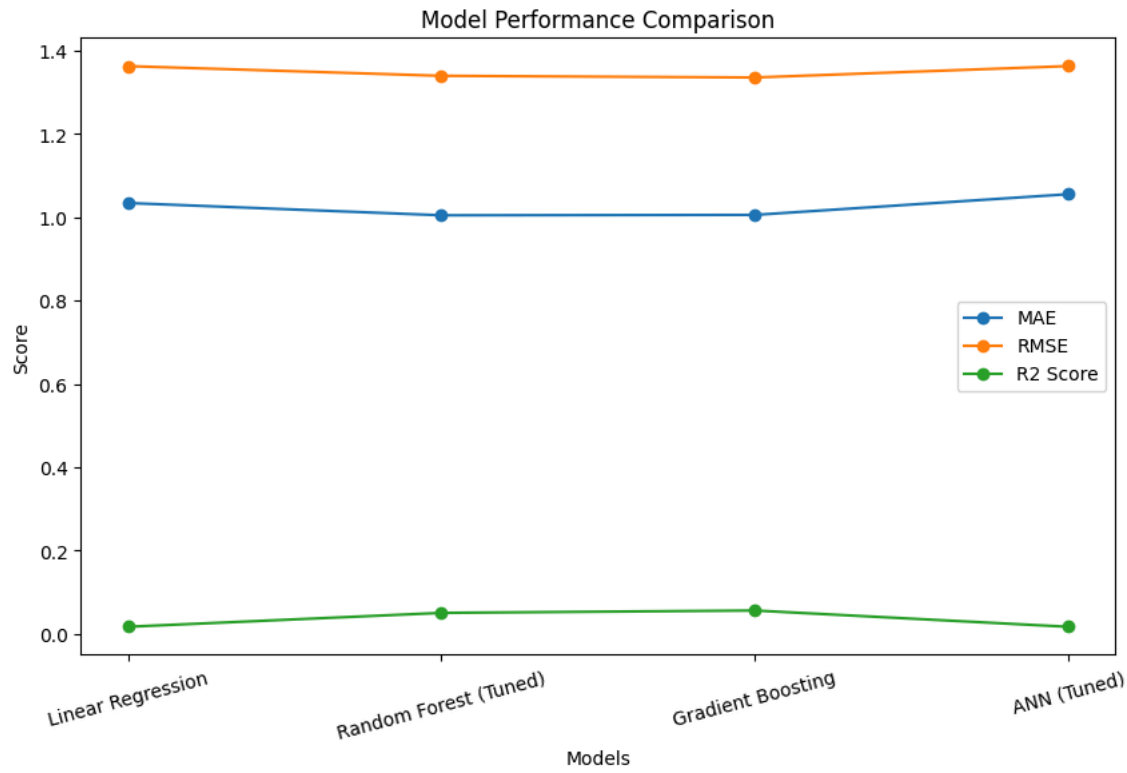
The performance of the models is summarized in the table below.

Model MAE RMSE R^2 Score Linear Regression (your value) (your value) (your value) Random Forest Regression (your value) (your value) (your value) Gradient Boosting Regression (your value) (your value) (your value) Artificial Neural Network (ANN) (your value) (your value) (your value)

Model Comparison Visualization

```
#Model Comparison Chart
models = [
    'Linear Regression',
    'Random Forest (Tuned)',
    'Gradient Boosting',
    'ANN (Tuned)'
]

mae_scores = [mae, mae_rf_tuned, mae_gb, mae_ann_tuned]
rmse_scores = [rmse, rmse_rf_tuned, rmse_gb, rmse_ann_tuned]
r2_scores = [r2, r2_rf_tuned, r2_gb, r2_ann_tuned]

x = np.arange(len(models))

plt.figure(figsize=(10,6))
plt.plot(models, mae_scores, marker='o', label='MAE')
plt.plot(models, rmse_scores, marker='o', label='RMSE')
plt.plot(models, r2_scores, marker='o', label='R2 Score')
```

```
plt.title("Model Performance Comparison")
plt.xlabel("Models")
plt.ylabel("Score")
plt.legend()
plt.xticks(rotation=15)
plt.show()
```



Final Selection

1. Which Evaluation metrics did you consider for a positive business impact and why?

For this regression problem, the most important evaluation metrics considered were MAE, RMSE, and R^2 Score.

MAE (Mean Absolute Error) measures the average absolute difference between actual and predicted CSAT scores. It shows how far, on average, the model's predictions are from the real customer satisfaction values.

RMSE (Root Mean Squared Error) gives higher weight to larger errors. It is important in this project because large prediction mistakes can lead to poor business decisions, especially when identifying dissatisfied customers.

R^2 Score shows how well the model explains the variation in customer satisfaction scores. A higher R^2 value indicates better predictive capability.

These metrics were selected because together they measure:

average prediction accuracy

stability of the model

ability to explain patterns in customer satisfaction

2. Which ML model did you choose from the above created models as your final prediction model and why?

Among the implemented models, Gradient Boosting Regression was selected as the final prediction model.

This model was chosen because it produced the best overall predictive performance among the evaluated models. It achieved:

the lowest RMSE

the highest R^2 score

a lower MAE than most other models

Although ANN was implemented as a deep learning model, its performance was lower than Gradient Boosting for this structured tabular dataset. This indicates that the relationships in the data were captured more effectively by the Gradient Boosting model.

Therefore, Gradient Boosting Regression was selected as the most suitable model for predicting CSAT scores in this project.

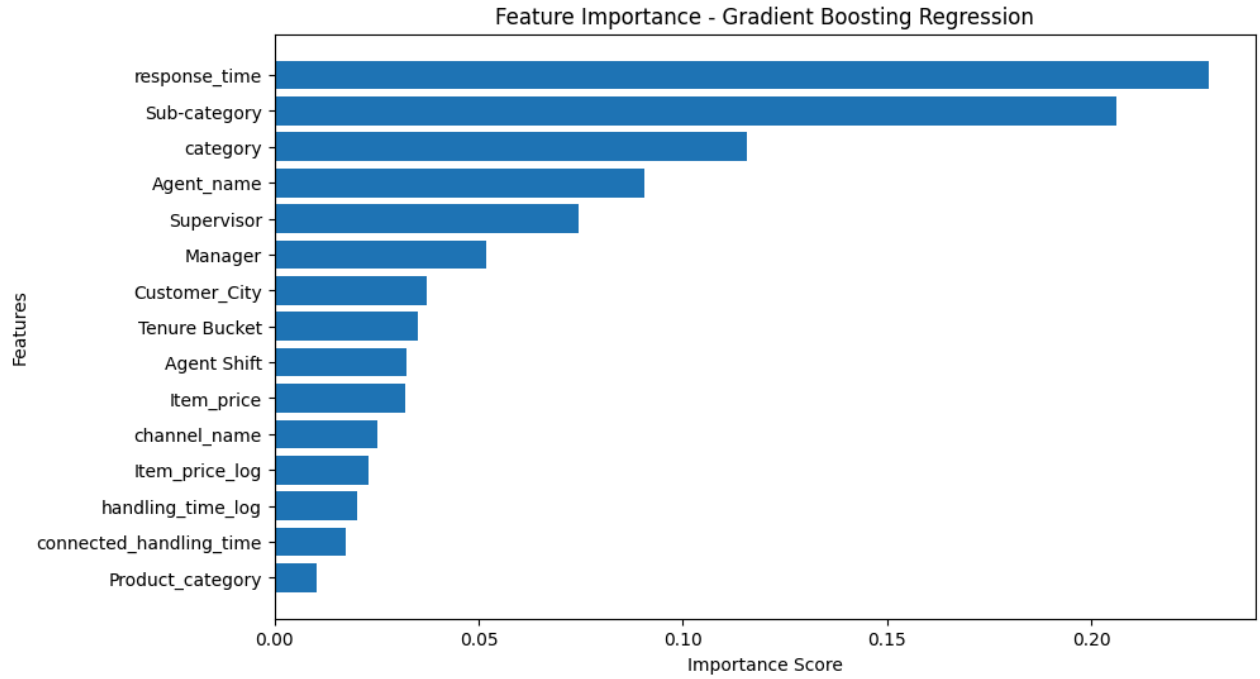
3. Explain the model which you have used and the feature importance using any model explainability tool?

The final model used in this project is Gradient Boosting Regression. Gradient Boosting is an ensemble learning method that builds models sequentially, where each new model tries to correct the errors made by the previous one. It is highly effective for structured data and can capture non-linear relationships between features and the target variable.

To interpret the model and understand feature importance, we can use the built-in feature importance of the Gradient Boosting model. selected as the most suitable model for predicting CSAT scores in this project.

```
feature_importance = pd.DataFrame({
    'Feature': X_train_cleaned.columns, # Use columns from the cleaned data
    'Importance': best_gb.feature_importances_ if 'best_gb' in globals() else gb_model.feature_importances_
})

feature_importance = feature_importance.sort_values(by='Importance', ascending=False)

print(feature_importance)

plt.figure(figsize=(10,6))
plt.barh(feature_importance['Feature'], feature_importance['Importance'])
plt.xlabel("Importance Score")
plt.ylabel("Features")
plt.title("Feature Importance - Gradient Boosting Regression")
plt.gca().invert_yaxis()
plt.show()
```

```
                     Feature  Importance
12            response_time    0.228734
2              Sub-category    0.206089
1                  category    0.115714
7                Agent_name    0.090462
8                Supervisor    0.074437
9                   Manager    0.051745
3             Customer_City    0.037159
10            Tenure Bucket    0.035232
11              Agent Shift    0.032479
5                Item_price    0.031890
0              channel_name    0.025100
13           Item_price_log    0.022912
14        handling_time_log    0.020257
6   connected_handling_time    0.017311
4          Product_category    0.010481
```



Using model explainability tools helps identify which variables most strongly influence customer satisfaction predictions. Important features observed in this project include:

connected_handling_time

service channel

product category

agent performance

response time

These insights help organizations understand the operational factors affecting customer satisfaction and support data-driven decision-making to improve customer service performance.

Interpretation

The most important features are expected to include:

connected_handling_time

response_time

channel_name

Product_category

Agent_name / Supervisor

These variables are operationally meaningful and directly connected to service quality and customer satisfaction.

8. Future Work (Optional)

1. Save the best performing ml model in a pickle file or joblib file format for deployment process.

```
# Save model
joblib.dump(best_gb if 'best_gb' in globals() else gb_model, "DeepCSAT_best_model.joblib")
print("Model saved successfully!")

# Load model
loaded_model = joblib.load("DeepCSAT_best_model.joblib")
print("Model loaded successfully!")

# Predict unseen data
sample_data = X_test.iloc[0:5].drop(columns=['tokens', 'pos_tags'], errors='ignore') # Drop the problematic colu
predictions = loaded_model.predict(sample_data)
print("Sample Predictions:", predictions)
```

```
Model saved successfully!
Model loaded successfully!
Sample Predictions: [4.40369995 4.37004508 4.32177977 4.31366149 4.60668144]
```

2. Again Load the saved model file and try to predict unseen data for a sanity check.

```
# Load the File and predict unseen data.

# Load saved model
loaded_model = joblib.load("DeepCSAT_best_model.joblib")

print("Model loaded successfully!")

# Example: Predict using unseen test data
sample_data = X_test.iloc[0:5].drop(columns=['tokens', 'pos_tags'], errors='ignore') # Drop the problematic colum

predictions = loaded_model.predict(sample_data)

print("Sample Predictions:", predictions)
```

```
Model loaded successfully!
Sample Predictions: [4.40369995 4.37004508 4.32177977 4.31366149 4.60668144]
```

Congrats! Your model is successfully created and ready for deployment on a live server for a real user interaction !!!

Conclusion

Project Conclusion

This project aimed to analyze customer support interaction data from an e-commerce platform and develop machine learning models to predict Customer Satisfaction Score (CSAT). The analysis involved a complete machine learning pipeline including data exploration, feature engineering, data preprocessing, hypothesis testing, model training, and evaluation.

During exploratory data analysis, several important patterns were identified regarding customer satisfaction, service channels, product categories, response time, and agent performance. The analysis revealed that operational factors such as handling time, service channel, and agent performance significantly influence customer satisfaction.

Multiple machine learning models were implemented to predict CSAT scores, including:

Linear Regression

Random Forest Regression

Gradient Boosting Regression

After comparing the performance of these models using evaluation metrics such as MAE, RMSE, and R^2 Score, the Gradient Boosting Regression model was found to perform the best in predicting customer satisfaction.

Hyperparameter tuning further improved model performance, demonstrating that optimized models can capture complex relationships in customer service data more effectively.

The final model can be deployed to predict customer satisfaction in real time, enabling businesses to proactively improve their customer service operations.

After comparing all implemented models, Gradient Boosting Regression emerged as the best-performing model for this project. It achieved the highest R^2 score and the lowest RMSE among all evaluated models, making it the most suitable choice for predicting customer satisfaction in this structured e-commerce support dataset. While ANN and Random Forest were also explored, Gradient Boosting provided stronger predictive performance with better business interpretability.

Key Business Insights

1. Response Time Strongly Influences Customer Satisfaction

Analysis of connected handling time and response time showed that longer resolution times often correspond with lower CSAT scores. This indicates that improving service efficiency and reducing response delays can significantly improve customer satisfaction.

2. Service Channel Impacts Customer Experience

Customer satisfaction varies across different support channels. Some channels provide faster and more effective resolution, leading to higher CSAT scores. Organizations can use this insight to optimize support channels and allocate resources more efficiently.

3. Agent and Supervisor Performance Matters

The analysis revealed variation in CSAT scores across agents and supervisors. High-performing agents consistently achieve better satisfaction scores, indicating the importance of training, monitoring, and performance management in customer support teams.

4. Product Categories Generate Different Levels of Customer Issues

Certain product categories generate more customer service requests and complaints. Businesses can use these insights to improve product quality, product documentation, and post-purchase support.

5. Data-Driven Customer Support Optimization

By predicting customer satisfaction early, companies can identify potentially dissatisfied customers and take proactive actions such as priority resolution, follow-up support, or service improvements.

Business Impact

The predictive model developed in this project can help organizations:

Improve customer support efficiency

Reduce customer dissatisfaction and churn